

หัวข้อโครงการ	AI ดันเจี้ยนมาสเตอร์สำหรับการเล่นเทเบิลท็อป RPG แบบแคชวล
หน่วยกิต	6
ผู้เขียน	นายภนลภัส สุทธิมาลา
อาจารย์ที่ปรึกษา	ผศ.ดร. สุริยา นักสุภักพงศ์ ดร. ไพสิฐ ชันอาสา
หลักสูตร	วิศวกรรมศาสตรบัณฑิต
สาขาวิชา	วิศวกรรมหุ่นยนต์และระบบอัตโนมัติ
คณะ	สถาบันวิทยาการหุ่นยนต์ภาคสนาม
ปีการศึกษา	2568

บทคัดย่อ

งานวิจัยนี้มีวัตถุประสงค์เพื่อพัฒนาต้นแบบระบบปัญญาประดิษฐ์ที่ทำหน้าที่เป็น Dungeon Master (AI DM) สำหรับการเล่นเกม Tabletop RPG ในกลุ่มผู้เล่นแคชวล เพื่อลดข้อจำกัดในการที่ต้องพึ่งพา Dungeon Master ที่เป็นมนุษย์ซึ่งต้องอาศัยทั้งความรู้ความเข้าใจในกฎกติกาและการเตรียมตัวล่วงหน้าเป็นอย่างมาก ระบบที่พัฒนาขึ้นจะมุ่งเน้นการสร้างประสบการณ์การเล่นที่เข้าถึงง่ายและเป็นธรรมชาติ โดยจะประกอบด้วยกระบวนการหลักคือ การประมวลผลคำสั่งร่วมกับแบบจำลองภาษาขนาดใหญ่ (LLM) เพื่อสร้างสรรค์เนื้อเรื่องและสถานการณ์ในเกม ระบบดังกล่าวจะถูกกำกับด้วย Rules Engine ที่ออกแบบตามกติกา “Lite 5e” เพื่อรักษาความสมดุลและความถูกต้องของกลไกเกม พร้อมทั้งมีระบบ Game State Tracker สำหรับติดตามสถานะต่างๆ ของผู้เล่นและเกมอย่างต่อเนื่อง เช่น ค่าพลังชีวิต ไอเทม และตำแหน่ง นอกจากนี้ ระบบจะแสดงผลผ่านส่วนต่อประสานกับผู้ใช้ (UI) ที่เรียบง่ายซึ่งออกแบบมาเพื่อการใช้งานบนอุปกรณ์เดียว พร้อมฟังก์ชันเสริม (Optional Functions) ในการรับคำสั่งเสียง (ASR) และการอ่านออกเสียงคำบรรยาย (TTS) เพื่อเป็นทางเลือกในการโต้ตอบ

คำสำคัญ : เอไอ ดันเจี้ยนมาสเตอร์ / เทเบิลท็อปอาร์พีจี / แบบจำลองภาษาขนาดใหญ่ (LLM) / การรู้จำเสียงพูดอัตโนมัติ (ASR) / การแปลงข้อความเป็นเสียง (TTS) / เอ็นจิน์กฎกติกา / ระบบติดตามสถานะเกม / การประมวลผลภาษาธรรมชาติ

Project Title	AI Dungeon Master for Casual Tabletop RPG Play
Project Credits	6
Candidate	Mr. Pollapaat Suttimala
Project Advisor	Asst.Prof.Dr. Suriya Natsupakpong Dr. Paisit Khanarsa
Program	Bachelor of Engineering
Field of Study	Robotics and Automation Engineering
Faculty	Institute of Field Robotics
Academic Year	2025

Abstract

This project aims to develop a prototype Artificial Intelligence system that functions as a Dungeon Master (AI DM) for casual Tabletop RPG players. The primary goal is to mitigate the constraints of relying on a human Dungeon Master, which requires extensive rule knowledge and significant preparation time. The developed system focuses on creating an accessible and natural gameplay experience. The core process involves processing commands in conjunction with a Large Language Model (LLM) to generate narrative content and in-game scenarios. This system is governed by a Rules Engine designed according to "Lite 5e" rules to maintain game balance and mechanical accuracy. Additionally, a Game State Tracker continuously monitors various player and game statuses, such as hit points, items, and location. Finally, the system presents information through a simple User Interface (UI) designed for single-device usability, which also includes optional functions for Automatic Speech Recognition (ASR) and Text-to-Speech (TTS) as alternative interaction methods.

Keywords : AI Dungeon Master / Tabletop RPG / Large Language Model (LLM) / Automatic Speech Recognition (ASR) / Text-to-Speech (TTS) / Rules Engine / Game State Tracker / Natural Language Processing

บทที่ 1 บทนำ

1.1 ความสำคัญและที่มาของงานวิจัย

ในช่วงไม่กี่ปีที่ผ่านมา กระแสความนิยมในเกมกระดาน (Board Game) และเกมสวมบทบาทบนโต๊ะ (Tabletop Role-Playing Game หรือ Tabletop RPG) ได้รับความสนใจเพิ่มขึ้นอย่างกว้างขวาง โดยเฉพาะเกมอย่าง Dungeons & Dragons (D&D) ซึ่งเป็นเกมที่ผู้เล่นกลุ่มหนึ่งจะสวมบทบาทเป็นตัวละครนักผจญภัย และดำเนินเรื่องราวตามจินตนาการร่วมกัน โดยมีผู้เล่นอีกคนหนึ่งทำหน้าที่เป็น "ดันเจียนมาสเตอร์" (Dungeon Master หรือ DM) คอยสร้างโลก บรรยายสถานการณ์ และควบคุมตัวละครอื่นๆ การรับรู้ที่เพิ่มขึ้นผ่านสื่อออนไลน์และผู้มีอิทธิพลในวงการเกม ได้ดึงดูดให้มีผู้เล่นหน้าใหม่และกลุ่มผู้เล่นทั่วไป (Casual Player) จำนวนมากที่ต้องการสัมผัสประสบการณ์การเล่าเรื่องและการผจญภัยอันเป็นเอกลักษณ์ของเกมประเภทนี้

อย่างไรก็ตาม การเข้าถึงประสบการณ์การเล่น Tabletop RPG ยังคงมีอุปสรรคสำคัญอยู่ที่บทบาทของดันเจียนมาสเตอร์ ซึ่งเปรียบเสมือนหัวใจหลักของเกม การทำหน้าที่ DM ให้สำเร็จลุล่วงได้นั้นต้องอาศัยทักษะและความทุ่มเทหลายด้าน ไม่ว่าจะเป็นความเข้าใจในกฎกติกาที่ซับซ้อน, ความคิดสร้างสรรค์ในการค้นสดและปรับเปลี่ยนเนื้อเรื่องตามการกระทำของผู้เล่น, และที่สำคัญที่สุดคือเวลาในการเตรียมการล่วงหน้าเป็นอย่างมาก ตั้งแต่การสร้างแผนที่ การออกแบบสถานการณ์ ไปจนถึงการเตรียมข้อมูลของศัตรูและตัวละครต่างๆ ซึ่งอาจใช้เวลาเตรียมการหลายวันสำหรับเซสชันการเล่นเพียงครั้งเดียว ข้อจำกัดเหล่านี้ทำให้การเล่นในลักษณะที่ไม่ได้นัดหมายล่วงหน้า หรือการเล่นในบรรยากาศสบายๆ เช่น ที่ร้านกาแฟหรือในงานสังสรรค์ ซึ่งเป็นที่ต้องการของผู้เล่นกลุ่มแคชชวล เป็นไปได้ยากอย่างยิ่ง

แม้ว่าในปัจจุบันจะมีเครื่องมือที่ใช้ปัญญาประดิษฐ์ (AI) เข้ามาเป็นทางเลือก เช่น แอปพลิเคชันช่วยเล่าเรื่องอย่าง AI Dungeon หรือการสนทนากับแชทบอททั่วไป แต่เครื่องมือเหล่านี้ยังเป็นเพียงผู้ช่วยเล่าเรื่องที่เน้นการสร้างบทสนทนาโต้ตอบเท่านั้น และยังไม่สามารถมอบประสบการณ์การเล่น "เกม" ได้อย่างสมบูรณ์ เนื่องจากขาดองค์ประกอบหลักที่สำคัญ ได้แก่ การบังคับใช้กฎกติกาเพื่อรักษาความสมดุลและความท้าทาย, กลไกการทอยลูกเต๋าเพื่อวัดผลการกระทำ, และระบบติดตามสถานะของเกม (Game State) ที่ชัดเจน เช่น ค่าพลังชีวิต ไอเทม และตำแหน่งของตัวละคร ส่งผลให้ผลลัพธ์ที่ได้ใกล้เคียงกับการสนทนาโต้ตอบทั่วไปมากกว่าการเล่นเกม นอกจากนี้ ระบบดังกล่าวยังมีข้อจำกัดด้านการจดจำบริบทและความต่อเนื่องของเนื้อเรื่องในระยะยาว (Long-term Memory) ทำให้ไม่สามารถรักษาสถานะของเกมได้อย่างน่าเชื่อถือ

ดังนั้น โครงการนี้จึงมีเป้าหมายในการพัฒนาต้นแบบระบบ "ปัญญาประดิษฐ์ต้นเจียนมาสเตอร์" (AI Dungeon Master) สำหรับผู้เล่นกลุ่มแคชชวลโดยเฉพาะ เพื่อลดช่องว่างและอุปสรรคดังกล่าว โดยเป็นการบูรณาการเทคโนโลยีหลายส่วนเข้าด้วยกันเพื่อเอาชนะข้อจำกัดของเครื่องมือที่มีอยู่เดิม หัวใจหลักของระบบคือการผสานการทำงานระหว่างแบบจำลองภาษาขนาดใหญ่ (LLM) ซึ่งทำหน้าที่สร้างสรรค์เนื้อเรื่อง กับ "เอนจินกฎกติกา" (Rules Engine) ที่สร้างขึ้นตามกฎ "Lite 5e" (ดูภาคผนวก ก) เพื่อควบคุมความถูกต้องและความสมดุลของกลไกเกม ระบบจะมี "ระบบติดตามสถานะเกม" (Game State Tracker) เพื่อจัดการข้อมูลสำคัญอย่างต่อเนื่อง และนำเสนอผ่านส่วนต่อประสานกับผู้ใช้ (UI) ที่เรียบง่าย นอกจากนี้ ยังมีการผนวกเทคโนโลยีสนับสนุน เช่น ระบบการรู้จำเสียงพูด (ASR) เพื่อเป็นทางเลือกในการโต้ตอบ เป้าหมายสูงสุดของโครงการคือการลดการพึ่งพา Dungeon Master ที่เป็นมนุษย์ แต่ยังคงรักษาแก่นแท้ของความสมดุล ความท้าทาย และความสนุกของการเล่น Tabletop RPG ไว้ได้อย่างครบถ้วน

1.2 วัตถุประสงค์ของงานวิจัย

1. เพื่อพัฒนาต้นแบบระบบ AI Dungeon Master ที่ใช้แบบจำลองภาษาขนาดใหญ่ (LLM) ในการดำเนินเรื่องราวและเหตุการณ์ ภายใต้การกำกับของ Rules Engine ที่แม่นยำตามกติกาแบบ Lite 5e
2. เพื่อออกแบบและพัฒนา Rules Engine และ Game State Tracker ที่สามารถบังคับใช้และติดตามกลไกหลักของเกมได้อย่างถูกต้องและต่อเนื่อง เช่น การตรวจสอบค่าความสามารถ, การโจมตี, ค่าความเสียหาย, พลังชีวิต และสถานะต่างๆ ของตัวละคร
3. เพื่อพัฒนาส่วนต่อประสานกับผู้ใช้ (UI) ที่ใช้งานง่ายและตอบสนองต่อการเล่นจริง ประกอบด้วยหน้าต่างแสดงข้อมูลตัวละคร (Character Sheet), แผนที่ (Map), และบันทึกเหตุการณ์ (Log) เพื่อรองรับการเล่นบนอุปกรณ์เดียว รวมถึงบูรณาการเทคโนโลยีสนับสนุน (ASR และ TTS) เพื่อเป็นทางเลือกในการโต้ตอบกับระบบ

1.3 ประโยชน์และผลที่คาดว่าจะได้รับจากงานวิจัย

1. ได้รับต้นแบบระบบ AI Dungeon Master ที่สามารถใช้งานได้จริงสำหรับผู้เล่นกลุ่มแคชชวล ช่วยลดอุปสรรคในการเริ่มต้นเล่น Tabletop RPG
2. ได้รับแนวทางการบูรณาการเทคโนโลยีแบบจำลองภาษาขนาดใหญ่ (LLM) เข้ากับระบบเชิงตรรกะ (Rules Engine, State Tracker) สำหรับประยุกต์ใช้กับเกมที่มีโครงสร้างและกฎเกณฑ์ชัดเจน รวมถึงแนวทางการผนวกเทคโนโลยีสนับสนุน (ASR, TTS) เข้ากับระบบ

- 3. ได้รับระบบที่ช่วยยกระดับประสบการณ์การเล่นในบริบทที่ไม่เป็นทางการให้สะดวกและเข้าถึงง่ายยิ่งขึ้น
- 4. ได้รับองค์ความรู้เชิงวิศวกรรมที่เป็นรากฐานสำคัญในการต่อยอดโครงการในอนาคต เช่น การพัฒนาให้รองรับการเล่นหลายคน (Multiplayer) ผ่านเครือข่าย หรือการพัฒนาให้รองรับภาษาไทย

1.4 ขอบเขตงานวิจัย

1.4.1 ด้านระบบเกมและกติกา

1. ระบบรองรับการเล่น Tabletop RPG โดยใช้ชุดกติกาแบบ "Lite 5e" ซึ่งเป็นกฎที่คณะผู้จัดทำได้นิยามและกำหนดขอบเขตขึ้นโดยเฉพาะ (ดูรายละเอียดฉบับเต็มในภาคผนวก ก) โดยได้ปรับลดความซับซ้อนลง แต่ยังคงรักษากลไกหลักที่จำเป็นไว้ เช่น การตรวจสอบค่าความสามารถ (d20 check), การโจมตีและความเสียหาย, ค่าพลังชีวิต (HP), และการเคลื่อนที่แบบโซน (Zone-based Movement) ขอบเขตของกลไกเกมที่ระบบรองรับจะจำกัดอยู่ภายใต้กฎ Lite 5e นี้เท่านั้น เพราะ LLM-only มีปัญหาในการ "แหกกฎ" (ตามที่วิเคราะห์ในบทที่ 2), ดังนั้น ทางออกแรกของเราคือการ "ลดความซับซ้อน" (Simplify) ของกฎที่ AI ต้องจำลอง

Feature	Standard D&D 5e	Our "Lite 5e"
Rulebook	300+ หน้า (Player's Handbook)	~3-4 หน้า (ในภาคผนวก ก)
Stats (ค่าสถานะ)	6 ค่า (STR, DEX, CON, INT, WIS, CHA)	3 ค่า (PHYS, MENT, SOC)
Map (แผนที่)	ตาราง Grid ซับซ้อน (5ft. squares)	"Zone-based" (ใกล้, กลาง, ไกล)
Skills (ทักษะ)	18 ทักษะ (เช่น Athletics, Stealth, Arcana)	รวมอยู่ใน 3 ค่าสถานะหลัก (เช่น ปีนป่าย = PHYS, สังเกต = MENT)

ตารางที่ X การเปรียบเทียบขอบเขต D&D 5e มาตรฐาน และ Lite 5e

- 2. ระบบถูกออกแบบมาเพื่อรองรับการเล่นเนื้อเรื่องสั้นที่จบในตัว (One-shot Session) และมีระบบบันทึกสถานะเกม (Save/Load) เพียงหนึ่งช่องสำหรับเล่นต่อเนื่อง
- 3. รูปแบบการเล่นและการตัดสินผล ระบบนี้ใช้สถาปัตยกรรมแบบ 'Hybrid-Arbitrated' ซึ่งแบ่งการประมวลผลคำสั่งของผู้เล่นเป็น 2 ประเภทหลัก เพื่อให้มีความยืดหยุ่นเหมือนเล่นกับมนุษย์ แต่ยังคงความแม่นยำของกฎไว้
 - 1) การกระทำแบบมีกฎตายตัว (Fixed Actions): คือการกระทำที่มีกลไกชัดเจนใน Lite 5e Rulebook (ภาคผนวก ก) เช่น 'การโจมตี' (Attacking), 'การเคลื่อนที่' (Movement), หรือ 'การใช้ความสามารถ' (Use Ability) คำสั่งเหล่านี้จะถูก Rules Engine (ส่วนที่เป็นโค้ด) ประมวลผลก่อน เพื่อคำนวณผลลัพธ์ (เช่น Hit/Miss, Damage) จากนั้น LLM จะทำหน้าที่บรรยายผลลัพธ์ ที่คำนวณได้

2) การกระทำเชิงสร้างสรรค์ (Creative Actions): คือการกระทำที่อยู่นอกเหนือกฎตายตัวที่ระบุไว้ชัดเจน เช่น 'การสำรวจสภาพแวดล้อม' (Exploration), 'การไขปริศนา' (Puzzles), หรือการกระทำที่ผู้เล่นคิดขึ้นเอง (เช่น "เผาหีบเพื่อละลายกุญแจ") คำสั่งเหล่านี้จะถูกส่งให้ LLM (ในฐานะ DM) ตีความและทำการตัดสินใจก่อน (Arbitration) เช่น กำหนดค่าความยาก (DC) หรือระบุค่าสถานะ (PHYS, MENT, SOC) ที่ต้องใช้ในการทอยลูกเต๋า จากนั้น Rules Engine จึงจะทำการทอยลูกเต๋า ตามที่ LLM กำหนด และ LLM จะบรรยายผลลัพธ์ ตามผลการทอยนั้น

4. ขอบเขตการทำงานของต้นแบบ (Prototype Scope): เพื่อให้สามารถพัฒนาได้ทันตามกำหนดเวลา และตอบสนองต่อข้อเสนอแนะในการลดขอบเขต โครงการนี้จะมุ่งเน้นการพัฒนาตามขอบเขตดังตารางต่อไปนี้

In-Scope (สิ่งที่ทำในโครงการนี้)	Out-of-Scope (สิ่งที่ยังไม่ทำในโครงการนี้)
1. Fixed Actions: กลไกการต่อสู้ (Combat) และการกระทำหลักตามที่ระบุใน Lite 5e Rulebook (ภาคผนวก ก) อย่างครบถ้วน	1. Complex Rules: กฎกติกาที่ซับซ้อนนอกเหนือจาก Lite 5e (เช่น กฎการหลบเร้นขั้นสูง, การโจมตีด้วยโอกาส, สภาวะผิดปกติอื่นๆ นอกเหนือจากที่ระบุ)
2. Creative Actions: การสำรวจ (Exploration) และการไขปริศนา (Puzzles) ที่สามารถตัดสินใจผลได้ด้วยการทอย 3 ค่าสถานะหลัก (PHYS, MENT, SOC) ตามที่ LLM กำหนด DC	2. Complex Scenarios: ปริศนาแบบหลายขั้นตอน (Multi-stage puzzles), บทสนทนาที่แตกแขนง (Branching dialogues) ที่ซับซ้อน, หรือการสืบสวนสอบสวน
3. State Tracking: ติดตามเฉพาะสถานะที่จำเป็นต่อการเล่นตามกฎ Lite 5e (HP, Zone, Items พื้นฐาน, สถานะผิดปกติที่ระบุไว้)	3. Unforeseen Events: เหตุการณ์สุ่ม (Random Events), Quick-Time Events, หรือกลไกที่เกิดขึ้นนอกกรอบการเล่นของผู้เล่น
4. Scenario Content: รองรับการเล่นใน 1 สถานการณ์ทดสอบหลัก (Pilot Test Scenario) ที่ออกแบบไว้ล่วงหน้า (ลักษณะ Mission-based)	4. Dynamic World: โลกที่เปลี่ยนแปลงตลอดเวลา, NPC ที่มีความสัมพันธ์ซับซ้อน, หรือระบบเศรษฐกิจภายในเกม

ตารางที่ X ขอบเขตการทำงานของต้นแบบ

1.4.2 ด้านการปฏิสัมพันธ์กับผู้ใช้

1. ระบบรองรับอินพุตหลักผ่านการพิมพ์ (Text Input) และมีอินพุตทางเลือก (Optional Input) เป็นคำสั่งเสียงภาษาอังกฤษ ซึ่งจะถูกแปลงเป็นข้อความผ่านโมดูล ASR

2. ส่วนต่อประสานกับผู้ใช้ (UI) ถูกออกแบบมาเพื่อใช้งานบนอุปกรณ์เดียว และมีองค์ประกอบหลักดังนี้

- 1) การตั้งค่าเริ่มต้น ผู้เล่นสามารถกำหนดจำนวนตัวละคร (1-4 ตัว) และเลือกธีมของโลก/เนื้อเรื่อง (เช่น แฟนตาซี, ไซไฟ) ก่อนเริ่มเกม
- 2) การควบคุมตัวละคร มีกลไกให้ผู้ใช้เลือกตัวละครที่ต้องการจะสั่งการก่อนป้อนคำสั่ง (พูดหรือพิมพ์) เพื่อให้ระบบสามารถระบุผู้กระทำได้อย่างถูกต้อง
- 3) หน้าต่างแสดงข้อมูล (Character Sheet) แสดงค่าสถานะ, พลังชีวิต และความสามารถหลักของตัวละคร
- 4) แผนที่แบบโซน (Zone Map) แสดงตำแหน่งของผู้เล่นและศัตรูในลักษณะโซน (เช่น ระเบียงใกล้, กลาง, ไกล) แทนการใช้ตาราง
- 5) บันทึกเหตุการณ์ (Action Log) แสดงผลการกระทำ, ผลการทอยลูกเต๋า และการตัดสินใจตามกติกาของ Rules Engine อย่างโปร่งใส

1.4.3 ด้านเทคนิคและการวัดผล

1. ระบบจะใช้บริการ LLM ผ่าน API เป็นหลัก โดยไม่เน้นการพัฒนากราฟิกสามมิติหรือแอนิเมชันที่ซับซ้อน แต่ให้ความสำคัญกับความเสถียรและความเข้าใจง่ายในการเล่น
2. ความสำเร็จของต้นแบบจะถูกวัดผลผ่านเกณฑ์ชี้วัดเชิงหน้าที่ (Functional Criteria) ดังนี้
 - 1) ความถูกต้องของกฎ (Rule Adherence) Rules Engine ต้องสามารถตัดสินใจและคำนวณผลลัพธ์ตามกฎ Lite 5e ที่กำหนดไว้ได้อย่างถูกต้องแม่นยำ
 - 2) ความสมบูรณ์ของสถานะ (State Integrity) Game State Tracker ต้องสามารถบันทึกและปรับปรุงข้อมูลสถานะของเกมได้อย่างถูกต้อง ไม่เกิดข้อผิดพลาดร้ายแรงตลอดการเล่น
 - 3) การจัดการลำดับการเล่น (Flow Management): ระบบต้องสามารถนำเสนอทางเลือกและการกระทำที่ชัดเจน ทำให้ผู้เล่นสามารถดำเนินเกมไปในแต่ละรอบ (Turn) จนจบได้อย่างราบรื่นโดยไม่เกิดสถานะหยุดชะงักที่เกิดจากตัวระบบเอง

1.5 ข้อตกลงเบื้องต้น

1. ในการดำเนินโครงการนี้จะใช้ โมเดลโอเพนซอร์สสำเร็จรูป สำหรับระบบการรู้จำเสียงพูด (ASR) และใช้บริการ API (Application Programming Interface) จากผู้ให้บริการภายนอกสำหรับแบบจำลองภาษาขนาดใหญ่ (LLM) โดยจะเลือกใช้โมเดลที่มีค่าใช้จ่ายไม่สูง (เช่น Gemini Flash)
2. ผู้ใช้งานระบบจำเป็นต้องมี API Key ของตนเองสำหรับ LLM และรับผิดชอบค่าใช้จ่ายที่เกิดขึ้นจากการใช้บริการ API ดังกล่าว โดยใน UI จะมีช่องสำหรับผู้กรอก API Key

3. ต้นแบบของระบบที่พัฒนาขึ้นจะมุ่งเน้นประสบการณ์การเล่นบนอุปกรณ์เดียว (Single-device experience) โดยผู้เล่นหนึ่งคนสามารถควบคุมตัวละครได้หลายตัว หรือผู้เล่นหลายคนสามารถสลับกันเล่นได้
4. ชุดกติกา "Lite 5e" ที่ใช้ในโครงการจะเป็นชุดกติกาที่คณะผู้จัดทำได้นิยามและกำหนดขอบเขตขึ้นโดยอ้างอิงจากกลไกหลักของเกม Dungeons & Dragons ฉบับที่ 5

1.6 ขั้นตอนการดำเนินงาน

ในกระบวนการดำเนินงานวิจัยนี้ ได้ทำการแบ่งขั้นตอนการดำเนินงานเพื่อเพิ่มประสิทธิภาพของงาน โดยมีรายละเอียดดังนี้

1. ทบทวนทฤษฎีและงานวิจัยที่เกี่ยวข้องกับเทคโนโลยีที่จะนำมาใช้ในโครงการ
2. กำหนดคุณสมบัติของระบบที่ต้องการ (Requirements) และเกณฑ์ชี้วัดความสำเร็จ (Success Criteria)
3. ออกแบบสถาปัตยกรรมของระบบ (System Architecture) และแผนผังการไหลของข้อมูล (Data Flow)
4. พัฒนาด้านแบบขั้นที่หนึ่ง (Prototype-A) ซึ่งประกอบด้วยการเชื่อมต่อระบบ LLM, Rules Engine เบื้องต้น, และระบบบันทึกเหตุการณ์ (Log)
5. พัฒนา Rules Engine และ Game State Tracker
6. พัฒนาส่วนต่อประสานกับผู้ใช้ (UI) ซึ่งประกอบด้วย Character Sheet, Zone Map, และ Battle Log
7. บูรณาการทุกส่วนประกอบเป็นต้นแบบขั้นที่สอง (Prototype-B), ผนวกฟังก์ชันเสริม (ASR/TTS), และทำการทดสอบการทำงานเบื้องต้น
8. ทดลองใช้งานระบบ (Pilot Test) เพื่อรวบรวมผลตอบรับและข้อเสนอแนะสำหรับนำมาปรับปรุง
9. ปรับปรุงแก้ไขระบบขั้นสุดท้าย และจัดทำรายงานโครงการฉบับสมบูรณ์

1.7 นิยามศัพท์เฉพาะ

1. AI Dungeon Master (AI DM) หมายถึง ระบบปัญญาประดิษฐ์ที่ทำหน้าที่เป็นผู้ดำเนินเกมและผู้ตัดสินใจในเกม Tabletop RPG
2. Tabletop RPG (เทเบิลท็อปอาร์พีจี) หมายถึง เกมสวมบทบาทที่เล่นบนโต๊ะ โดยอาศัยการเล่าเรื่อง การทอยลูกเต๋า และจินตนาการของผู้เล่นเป็นหลัก
3. Large Language Model (LLM) หมายถึง แบบจำลองภาษาขนาดใหญ่ เป็นปัญญาประดิษฐ์ที่เชี่ยวชาญด้านการเข้าใจและสร้างข้อความที่เหมือนมนุษย์

4. Rules Engine (เอ็นจินกฎกติกา) หมายถึง ส่วนประกอบของซอฟต์แวร์ที่สร้างขึ้นเพื่อบังคับใช้กฎกติกาและกลไกของเกมโดยเฉพาะ
5. Game State Tracker (ระบบติดตามสถานะเกม) หมายถึง ส่วนประกอบของซอฟต์แวร์ที่ทำหน้าที่บันทึกและจัดการข้อมูลทั้งหมดที่เกิดขึ้นในเกมแบบเรียลไทม์
6. Lite 5e หมายถึง ชุดกติกาที่ถูกปรับลดความซับซ้อนลงจาก Dungeons & Dragons ฉบับที่ 5 เพื่อให้เหมาะกับผู้เล่นกลุ่มแคชวล
7. Automatic Speech Recognition (ASR) หมายถึง เทคโนโลยีการรู้จำเสียงพูดอัตโนมัติ ทำหน้าที่แปลงสัญญาณเสียงพูดให้เป็นข้อความ
8. Text-to-Speech (TTS) หมายถึง เทคโนโลยีการแปลงข้อความที่เป็นเสียงสังเคราะห์ ทำหน้าที่อ่านออกเสียงคำบรรยายของระบบเพื่อเป็นทางเลือกในการรับฟัง

บทที่ 2 ทฤษฎี/งานวิจัยที่เกี่ยวข้อง

ในบทนี้จะกล่าวถึงทฤษฎีพื้นฐานและเทคโนโลยีที่เกี่ยวข้องซึ่งเป็นหัวใจสำคัญในการพัฒนาโครงการ "AI Dungeon Master" รวมถึงทบทวนงานวิจัยและโครงการที่มีอยู่ก่อนหน้านี้ เพื่อวิเคราะห์แนวทางการพัฒนา จุดแข็ง จุดอ่อน และนำมาเป็นแนวทางในการออกแบบสถาปัตยกรรมของระบบในโครงการนี้ เนื้อหาจะแบ่งออกเป็นสองส่วนหลัก ได้แก่ ส่วนของทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง และส่วนของงานวิจัยที่เกี่ยวข้อง

2.1 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง

โครงการนี้เป็นการบูรณาการเทคโนโลยีปัญญาประดิษฐ์หลายแขนงเข้าด้วยกัน เพื่อสร้างระบบที่สามารถทำหน้าที่อันซับซ้อนของ Dungeon Master ได้ เทคโนโลยีหลักที่นำมาใช้ประกอบด้วย แบบจำลองภาษาขนาดใหญ่ (LLM) และสถาปัตยกรรมปัญญาประดิษฐ์แบบผสมผสาน (Hybrid AI Architecture) โดยมีเทคโนโลยีสนับสนุน (Supporting Technologies) ได้แก่ การรู้จำเสียงพูดอัตโนมัติ (ASR) และการแปลงข้อความเป็นเสียง (TTS)

2.1.1 แบบจำลองภาษาขนาดใหญ่ (Large Language Models - LLM)

แบบจำลองภาษาขนาดใหญ่ หรือ LLM คือแบบจำลองปัญญาประดิษฐ์ประเภทหนึ่งที่ได้รับการฝึกฝนด้วยข้อมูลข้อความจำนวนมาก ทำให้มีความสามารถสูงในการทำความเข้าใจและสร้างภาษาที่ใกล้เคียงกับมนุษย์ แบบจำลองที่มีชื่อเสียงในปัจจุบัน เช่น GPT (Generative Pre-trained Transformer) ของ OpenAI หรือ Gemini ของ Google ล้วนมีพื้นฐานมาจากสถาปัตยกรรมที่เรียกว่า Transformer ซึ่งเป็นปัจจัยสำคัญที่ทำให้ LLM สามารถจัดการกับความซับซ้อนของภาษาได้ดี

1. จุดแข็งสำหรับโครงการ

1) การสร้างสรรค์เนื้อหา (Creative Text Generation) จุดแข็งที่สำคัญที่สุดของ LLM คือความสามารถในการสร้างเรื่องราว, คำบรรยายฉาก, และบทสนทนาของตัวละคร (NPC) ได้อย่างสร้างสรรค์และต่อเนื่อง ทำให้โลกของเกมมีความสมจริงและน่าสนใจ การทำงานแบบ Probabilistic ซึ่งอิงตามรูปแบบภาษาที่ได้เรียนรู้มา ช่วยให้ LLM สามารถสร้างผลลัพธ์ที่หลากหลายและคาดไม่ถึง ซึ่งเป็นหัวใจของความสนุกในเกม Tabletop RPG ที่ผู้เล่นมักจะกระทำการนอกกรอบที่คาดไว้ ตัวอย่างเช่น หากผู้เล่นตัดสินใจปีนหน้าต่างของโรงเตี๊ยมแทนที่จะเดินเข้าประตูหน้า LLM สามารถ

สร้างคำบรรยายถึงสภาพของหน้าต่างที่เก่าแก่ เสียงไม้ที่ลั่นเอี๊ยด และลมเย็นที่พัดเข้ามาได้ในทันทีโดยไม่ต้องมีการเตรียมสคริปต์ไว้ล่วงหน้า

2) การจัดการบทสนทนา (Dialogue Management) LLM สามารถเข้าใจเจตนาของผู้เล่นจากคำสั่งที่เป็นภาษาธรรมชาติ และสร้างการตอบสนองที่สอดคล้องกับสถานการณ์ได้อย่างยืดหยุ่น ซึ่งแตกต่างอย่างสิ้นเชิงจากระบบบทสนทนาแบบต้นไม้ (Dialogue Tree) ในเกมดั้งเดิมที่ผู้เล่นถูกจำกัดให้เลือกตอบตามตัวเลือกที่กำหนดไว้ล่วงหน้าเท่านั้น ความสามารถนี้ทำให้ตัวละครในเกม (NPC) มีชีวิตชีวาและตอบสนองได้อย่างสมจริง ตัวอย่างเช่น แทนที่จะมีเพียงตัวเลือกสนทนา 3 ข้อ ผู้เล่นสามารถถามคำถามปลายเปิดกับ NPC เช่น "คุณเคยได้ยินข่าวลือเกี่ยวกับมังกรบนภูเขาบ้างไหม?" และ LLM ก็สามารถสร้างคำตอบที่สอดคล้องกับบทบาทและบุคลิกของ NPC ตัวนั้นได้ทันที

2. ข้อจำกัดและประเด็นที่ต้องพิจารณา

1) ภาวะไร้สถานะ (Statelessness) และข้อจำกัดของหน่วยความจำ: LLM ไม่มีหน่วยความจำถาวร มันจะจดจำข้อมูลได้เท่าที่มีอยู่ใน "หน้าต่างบริบท" (Context Window) เท่านั้น ซึ่งเป็นเหมือนหน่วยความจำระยะสั้นที่มีขนาดจำกัด เมื่อการสนทนาหรือเรื่องราวดำเนินไปนานขึ้น ข้อมูลในช่วงแรกๆ จะหลุดออกจากหน้าต่างบริบท ทำให้ LLM อาจ "ลืม" เหตุการณ์สำคัญ ชื่อตัวละคร หรือข้อมูลที่เคยเกิดขึ้นไปแล้วได้ ปัญหานี้เป็นอุปสรรคสำคัญต่อการสร้างเรื่องราวที่ต่อเนื่องและสมเหตุสมผลในระยะยาว

2) การยึดถือกฎกติกา (Rule Adherence): โดยธรรมชาติแล้ว LLM เป็นแบบจำลองเชิงความน่าจะเป็น (Probabilistic Model) ไม่ใช่ระบบเชิงตรรกะ (Logical System) ทำให้มันไม่สามารถรับประกันได้ว่าจะปฏิบัติตามกฎกติกาที่ซับซ้อนและตายตัวของเกมได้อย่างถูกต้อง 100% บ่อยครั้งที่ LLM อาจสร้างผลลัพธ์ที่ดูสมเหตุสมผลแต่ผิดหลักกลไกของเกม หรือที่เรียกว่า "Hallucination" ซึ่งเป็นสิ่งที่ไม่สามารถยอมรับได้ในบริบทของเกมที่ต้องการความยุติธรรม

ข้อจำกัดทั้งสองข้อนี้เป็นความท้าทายหลักเชิงสถาปัตยกรรม และเป็นเหตุผลสำคัญที่โครงการนี้จำเป็นต้องมีส่วนประกอบอื่นเข้ามาทำงานร่วมกับ LLM แทนที่จะใช้ LLM เพียงอย่างเดียว

2.1.2 การรู้จำเสียงพูดอัตโนมัติ (Automatic Speech Recognition - ASR)

ASR คือเทคโนโลยีที่ทำหน้าที่แปลงสัญญาณเสียงพูดของมนุษย์ให้กลายเป็นข้อมูลข้อความดิจิทัล เพื่อให้คอมพิวเตอร์สามารถนำไปประมวลผลต่อได้ ในบริบทของโครงการนี้ ASR จะถูกนำมาใช้เป็น "ฟังก์ชันเสริม" (Optional Function) เพื่อเป็น "ทางเลือก" ให้ผู้เล่นสามารถสั่งการตัวละครด้วยเสียงพูดได้ นอกเหนือจากการพิมพ์

ในปัจจุบัน เทคโนโลยี ASR สามารถแบ่งได้เป็น 2 ประเภทหลัก คือแบบที่อาศัยบริการคลาวด์ (Cloud-based API) และแบบโมเดลโอเพนซอร์สที่ประมวลผลบนอุปกรณ์ของผู้ใช้โดยตรง (On-device) ในปัจจุบันมีโมเดลโอเพนซอร์สประสิทธิภาพสูงที่ประมวลผลบนอุปกรณ์ (On-device) เช่น Whisper ของ OpenAI หรือ Vosk ซึ่งช่วยรักษาความเป็นส่วนตัวของข้อมูลเสียง ในโครงการนี้จะทบทวนแนวทางดังกล่าวเพื่อนำมาบูรณาการ (Integrate) เป็นฟังก์ชันเสริม โดยมุ่งเน้นที่การเชื่อมต่อเป็นทางเลือกให้ผู้ใช้ มากกว่าการวัดผลประสิทธิภาพของตัวโมเดล

2.1.3 การแปลงข้อความเป็นเสียง (Text-to-Speech - TTS)

เช่นเดียวกับ ASR, เทคโนโลยี TTS (Text-to-Speech) จะถูกนำมาใช้เป็นฟังก์ชันเสริม เพื่อเพิ่มอรรถรส (Immersion) ในการเล่น โดยระบบจะอ่านออกเสียงคำบรรยายของ AI DM เป็นทางเลือกให้ผู้เล่น โครงการนี้จะทบทวนไลบรารีมาตรฐานที่มีใน Flutter หรือโมเดลโอเพนซอร์สขนาดเล็ก ที่ใช้ทรัพยากรน้อย โดยเน้นการบูรณาการเป็นฟังก์ชันทางเลือกเท่านั้น ไม่ได้มุ่งเน้นการวัดผลประสิทธิภาพของเสียงสังเคราะห์

2.1.4 สถาปัตยกรรมปัญญาประดิษฐ์แบบผสมผสาน (Hybrid AI Architecture)

เพื่อแก้ไขข้อจำกัดของ LLM ที่กล่าวไปข้างต้น โครงการนี้จึงได้นำแนวคิดของ สถาปัตยกรรมปัญญาประดิษฐ์แบบผสมผสาน หรือที่รู้จักในอีกชื่อหนึ่งว่า Neuro-Symbolic AI มาใช้ แนวคิดนี้คือการนำระบบสองประเภทที่มีความถนัดต่างกันมาทำงานร่วมกัน เปรียบได้กับการทำงานของสมองมนุษย์สองส่วน (System 1 และ System 2 Thinking)

1. Neural Component (LLM) เปรียบเสมือน System 1 คือส่วนของโครงข่ายประสาทเทียมที่ทำงานอย่างรวดเร็ว เป็นธรรมชาติ และใช้สัญชาตญาณ ในที่นี้คือ LLM ซึ่งทำหน้าที่เกี่ยวกับความคิดสร้างสรรค์ การเข้าใจภาษา และการสร้างเนื้อหาเชิงพรรณนา
2. Symbolic Component (Code) เปรียบเสมือน System 2 คือส่วนที่ทำงานอย่างช้าๆ เป็นเหตุเป็นผล และอยู่บนพื้นฐานของตรรกะและกฎเกณฑ์ที่กำหนดไว้อย่างชัดเจน ในโครงการนี้หมายถึง Rules Engine และ Game State Tracker ที่เขียนขึ้นด้วยโค้ดโปรแกรม

สถาปัตยกรรมแบบผสมผสานนี้จะแบ่งหน้าที่ความรับผิดชอบอย่างชัดเจน

1. LLM รับผิดชอบในส่วนของการเล่าเรื่อง (Narrative) เช่น การบรรยายฉาก, การตอบโต้ของ NPC

2. Rules Engine & Game State Tracker รับผิดชอบในส่วนของ "กลไกเกมและหน่วยความจำ" (Mechanics & Memory) เช่น การคำนวณความเสียหาย, การตรวจสอบผลการทอยลูกเต๋า, และการบันทึกว่า "ตอนนี้ผู้เล่นมีพลังชีวิตเหลือเท่าไร"

ด้วยสถาปัตยกรรมนี้ ระบบจะให้ LLM ทำในสิ่งที่ถนัด และใช้โค้ดโปรแกรมที่ทำงานอย่างแม่นยำมาจัดการในส่วนที่ LLM ทำได้ไม่ดี ซึ่งเป็นแนวทางที่ได้รับการยอมรับอย่างกว้างขวางในการพัฒนาระบบ AI ที่ต้องการทั้งความคิดสร้างสรรค์และความน่าเชื่อถือ

2.1.5 การสร้างความรู้ด้วยการดึงข้อมูลมาเสริม (Retrieval-Augmented Generation - RAG)

RAG เป็นเทคนิคที่กำลังได้รับความนิยมอย่างสูงในการเพิ่มประสิทธิภาพและความน่าเชื่อถือให้กับ LLM หลักการของ RAG คือ แทนที่จะปล่อยให้ LLM ตอบคำถามจาก "ความรู้" ที่มีอยู่ภายในเพียงอย่างเดียว ระบบจะทำการ ดึงข้อมูลที่เกี่ยวข้อง จากแหล่งข้อมูลภายนอก (เช่น เอกสาร, ฐานข้อมูล) แล้วนำข้อมูลนั้นมาประกอบในคำสั่ง (Prompt) เพื่อให้ LLM ใช้เป็นบริบทในการสร้างคำตอบ ในโครงการนี้ RAG จะเป็นกลไกสำคัญที่เชื่อมต่อระหว่าง LLM และ Symbolic Component

1. การดึงกฎกติกา (Rule Retrieval): เมื่อผู้เล่นต้องการกระทำการบางอย่างที่เกี่ยวข้องกับกฎ เช่น "ร่ายเวทมนตร์ Fireball" ระบบ RAG สามารถดึงข้อมูลกฎของเวทมนตร์ Fireball จากฐานข้อมูล แล้วส่งไปพร้อมกับคำสั่งให้ LLM เพื่อให้มันบรรยายผลลัพธ์ได้ถูกต้องตามกฎ เช่น Damage: 8d6 fire damage, Range: 150 feet
2. การดึงสถานะของเกม (State Retrieval): ในทุกๆ การกระทำของผู้เล่น ระบบ RAG จะดึงข้อมูลสถานะล่าสุดของเกมจาก Game State Tracker (เช่น Player HP: 12/15, Goblin HP: 5/7) มาประกอบในคำสั่ง เพื่อให้ LLM "รับรู้" สถานการณ์ปัจจุบันและสร้างเนื้อเรื่องที่สอดคล้องกับสิ่งที่เกิดขึ้นจริงในเกม เช่น การบรรยายว่าก๊อบลิน "ดูบาดเจ็บสาหัส" เมื่อพลังชีวิตเหลือน้อย

การใช้เทคนิค RAG ทำให้เราสามารถแก้ปัญหา Long-term Memory ของ LLM ได้อย่างมีประสิทธิภาพ และยังเป็นกลไกสำคัญในการกำกับให้ LLM สร้างเนื้อหาที่สอดคล้องกับกฎและสถานะของเกมอยู่เสมอ

2.2 งานวิจัยและโครงการที่เกี่ยวข้อง

ในช่วงไม่กี่ปีที่ผ่านมา มีความพยายามในการนำแบบจำลองภาษาขนาดใหญ่ (LLM) มาประยุกต์ใช้เป็น Dungeon Master (DM) ในหลากหลายรูปแบบ การทบทวนและวิเคราะห์งานวิจัยเหล่านี้ช่วยให้เห็นถึง วัฒนาการของเทคโนโลยี สถาปัตยกรรม (Settings), ข้อสันนิษฐาน (Assumptions), และข้อจำกัด (Gaps) ซึ่งเป็นข้อมูลสำคัญในการออกแบบสถาปัตยกรรมของโครงการนี้

2.2.1 Triyason (2023) - การใช้ ChatGPT เป็น DM สำหรับผู้เล่นคนเดียว (The "LLM-only" Baseline)

งานวิจัยของ Triyason (2023) นับเป็นหนึ่งในงานวิจัยกลุ่มแรกที่ทดลองใช้ ChatGPT ในการดำเนินเกม D&D พบว่า LLM สามารถสร้างเรื่องราวที่ต่อเนื่องและตอบสนองต่อการกระทำของผู้เล่นได้ดีพอที่จะทำให้ผู้เล่นใหม่ได้สัมผัสกับประสบการณ์ D&D อย่างไรก็ตาม ข้อจำกัดที่พบคือการตอบสนองที่ล่าช้า ในบางครั้ง และการขาดการแสดงอารมณ์ร่วม (Limited Emotional Engagement) ซึ่งส่งผลต่อความอินของผู้เล่น

เพื่อทำความเข้าใจบริบทของงานวิจัยนี้อย่างลึกซึ้ง (Deep Dive) และเพื่อสนับสนุนการออกแบบ สถาปัตยกรรมของโครงการนี้ สามารถวิเคราะห์องค์ประกอบของงานวิจัยดังกล่าวได้ดังนี้

1. สถาปัตยกรรมและกติกาที่ใช้ (Their Setting)

1) Rulebook/Tech: งานวิจัยนี้ใช้กฎ Dungeons & Dragons 5e (D&D 5e) เป็นพื้นฐาน โดยนำเสนอในรูปแบบข้อความล้วน (Text-only)

2) Architecture: สถาปัตยกรรมทางเทคนิคเป็นแบบ Monolithic (LLM-only) คือใช้ ChatGPT เพียงตัวเดียว และอาศัยเทคนิค Prompt Engineering (การออกแบบคำสั่ง) ในการกำหนดบทบาทและควบคุมพฤติกรรมให้ AI ทำหน้าที่เป็น DM

3) Engine: ระบบนี้ไม่มี Custom Game Engine หรือ Rules Engine แยกต่างหาก การตัดสินใจผลและการปฏิบัติตามกฎ (Rule Adherence) ทั้งหมดขึ้นอยู่กับความรู้ภายใน (Internal Knowledge) ของ ChatGPT เท่านั้น ซึ่งเปรียบเสมือนการใช้งาน "ChatGPT-as-DM" ผ่านหน้าต่างสนทนาปกติ

2. ข้อสันนิษฐานและข้อจำกัด (Their Assumption)

1) Solo Player: งานวิจัยตั้งข้อสันนิษฐานหลักคือผู้เล่นเป็นแบบ Solo Player (เล่นคนเดียวกับ AI)

2) Text-only Constraints: มีข้อจำกัดด้านรูปแบบการเล่นที่เป็น Textual ทั้งหมด ไม่มีการใช้กราฟิก แผนที่ หรือระบบทอยลูกเต๋าจำลอง (Dice Simulator) ภายนอก

3) Out-of-the-box: AI ที่ใช้เป็นแบบ "Out-of-the-box" (ไม่ผ่านการ Fine-tune) โดยอาศัย Prompt Engineering เพียงอย่างเดียว

3. การเปรียบเทียบ (Their Comparison)

1) Benchmark: งานวิจัยทำการเปรียบเทียบประสิทธิภาพแบบตัวต่อตัว (Head-to-head) ระหว่าง AI Dungeon Master กับ Human Dungeon Master

2) Method: ผู้เข้าร่วมการทดลองจะได้เล่นเกมกับทั้งสองระบบ จากนั้นจึงทำแบบประเมิน (Score and Evaluate) เพื่อเปรียบเทียบประสบการณ์ที่ได้รับในด้านต่างๆ

4. ผลลัพธ์การทดลอง (Their Performance)

1) Positive: ผลลัพธ์แสดงให้เห็นว่า LLM สามารถสร้าง "Coherent Narratives" (เนื้อเรื่องที่ต่อเนื่อง สมเหตุสมผล) และ "Reacted Dynamically" (ตอบสนองต่อการกระทำของผู้เล่น) ได้ในระดับที่น่าพอใจ

2) Negative (Performance): ผู้เล่นรายงานข้อจำกัดด้าน "Delayed Responses" (การตอบสนองล่าช้า/Latency) และ "Limited Emotional Engagement" (การขาดอารมณ์ร่วม) ซึ่งส่งผลกระทบต่อ "Player Immersion" (ความรู้สึกดื่มด่ำในเกม)

3) Negative (Logic/Rules): เนื่องจากขาดระบบจัดการกฎภายนอก AI จึงประสบปัญหาในการ "Update the Game State Consistently" (การรักษาสถานะเกมให้สม่ำเสมอ) เช่น การลืมหาค่าพลังชีวิต (HP), ลืมไอเทม, หรือลืมหเหตุการณ์ที่เพิ่งเกิดขึ้น ซึ่งเป็นข้อจำกัดร้ายแรงของสถาปัตยกรรมแบบ LLM-only

5. บทเรียนที่นำมาปรับใช้ (What We Will Use)

1) การออกแบบ Prompt (Prompt Design) เป็นปัจจัยสำคัญในการควบคุมพฤติกรรมของ LLM

2) ที่สำคัญที่สุด: ข้อค้นพบของ Triyason (เรื่องความล้มเหลวในการจัดการ State และ Rules) ถูกนำมาใช้เป็น Problem Statement หลัก ของโครงการนี้ เพื่อเป็นเหตุผลสนับสนุน (Justification) ว่าทำไมเราจึงจำเป็นต้องพัฒนา Rules Engine และ Game State Tracker แยกต่างหาก แทนที่จะพึ่งพา LLM เพียงอย่างเดียว

6. ช่องว่างและข้อจำกัดของงานวิจัย (The Gap/Limit)

1) ระบบของ Triyason มีลักษณะ "Minimalist and Fragile" (เรียบง่ายแต่เปราะบาง) เนื่องจากขาดกลไกเกม (Mechanics) ที่ชัดเจนและไม่มีส่วนต่อประสานกับผู้ใช้ (UI) ทำให้ประสบการณ์การเล่นเป็นแบบ "Fast but Shallow" (เร็วแต่ตื้นเขิน)

2) Gap ที่โครงการนี้จะเติมเต็ม

(1) เพิ่ม Lite 5e Rules Engine และ Game State Tracker เพื่อแก้ปัญหา "Consistency" (ความถูกต้องแม่นยำของกฎและสถานะ)

(2) เพิ่ม UI (Character Sheets, Zone Maps) และ Voice Input (ASR) เพื่อให้ระบบเข้าถึงง่ายและใช้งานสะดวกกว่า (More Accessible) การพิมพ์ข้อความเพียงอย่างเดียว

7. แนวทางการประเมินผล (Their Evaluation)

1) เราจะนำแนวคิดการใช้ "Post-game Surveys" (แบบสอบถามหลังการเล่น) ของงานวิจัยนี้มาปรับใช้ (Adapt)

2) ในการทดสอบ Pilot Test (หัวข้อ 3.5) เราจะใช้คำถามที่วัดผลด้าน Immersion, Narrative Coherence, และ Rule Adherence (เช่น Q1, Q3, Q4, Q5) เพื่อตรวจสอบว่าสถาปัตยกรรมแบบ Hybrid ของเราสามารถแก้ไขปัญหาที่พบในงานวิจัยของ Triyason ได้จริงหรือไม่

2.2.2 Sakellaridis (2024) - การเปรียบเทียบ LLM ที่ผ่านการ Fine-tune กับ DM มนุษย์

งานวิจัยนี้ได้ดำเนินการศึกษาต่อยอดโดยการนำแบบจำลองภาษาขนาดใหญ่ (LLM) มาผ่านกระบวนการปรับแต่งละเอียด (Fine-tuning) ด้วยชุดข้อมูลบทสนทนาจากเกม D&D และข้อมูลกฎกติกาโดยเฉพาะ เพื่อเปรียบเทียบประสิทธิภาพการดำเนินเกมกับ Dungeon Master (DM) ที่เป็นมนุษย์ ผลการศึกษาเบื้องต้นพบประเด็นที่น่าสนใจคือ ผู้เล่นประเมินให้คะแนนความรู้สึกดื่มด่ำ (Immersion) ต่อคำบรรยายฉากของ AIDM ในระดับสูงเทียบเท่าหรือสูงกว่า DM มนุษย์ ซึ่งแสดงให้เห็นถึงศักยภาพของ LLM ในการสร้างสรรค์เนื้อหาเชิงพรรณนา อย่างไรก็ตาม ข้อจำกัดสำคัญที่พบคือ AI มักมีแนวโน้มที่จะดำเนินเรื่องไปในทิศทางที่กำหนดไว้ล่วงหน้าอย่างเคร่งครัด (Railroading) ขาดความยืดหยุ่นในการตอบสนองต่อการตัดสินใจนอกกรอบของผู้เล่น และยังไม่สามารถสร้างความท้าทายในเกมได้ดีพอ

เพื่อทำความเข้าใจบริบทของงานวิจัยอย่างลึกซึ้ง (Deep Dive) และเพื่อสนับสนุนการออกแบบสถาปัตยกรรมของโครงการนี้ สามารถวิเคราะห์องค์ประกอบของงานวิจัยดังกล่าวได้ดังนี้

1. สถาปัตยกรรมและกติกาที่ใช้ (Their Setting)

1) Rulebook/Tech: งานวิจัยนี้ใช้บริบทของ Dungeons & Dragons 5th Edition (D&D 5e) ในรูปแบบสภาพแวดล้อมเกมสวมบทบาทแบบข้อความ (Text-based RPG Environment)

2) Architecture: สถาปัตยกรรมทางเทคนิคใช้ ChatGPT (LLM-based agent) ที่ผ่านการ Fine-tune ด้วยชุดข้อมูล "Critical Role Transcripts" (บันทึกบทสนทนาจากการเล่น D&D จริง) เพื่อให้โมเดลเรียนรู้รูปแบบการเล่าเรื่องและบทบาทสมมติ

3) Knowledge: ระบบ AI สามารถเข้าถึงกฎ D&D 5e ฉบับเต็มและเนื้อหาของโมดูลการผจญภัยสำเร็จรูป (Adventure Module) เรื่อง "The Sunless Citadel" ซึ่งถูกป้อนให้ AI ใช้เป็นฐานข้อมูลอ้างอิง

2. ข้อสันนิษฐานและข้อจำกัด (Their Assumption)

- 1) Solo Play Scenario: การทดลองตั้งอยู่บนสมมติฐานการเล่นแบบผู้เล่นเดี่ยว (Solo Player)
- 2) Pre-written Adventure: AI จะดำเนินเรื่องตามบทการผจญภัยที่เขียนไว้ล่วงหน้า ("The Sunless Citadel") ไม่ใช่การสร้างเนื้อเรื่องใหม่แบบสุ่ม (Procedural Generation)
- 3) Fine-tuning Hypothesis: มีข้อสันนิษฐานว่าการ Fine-tune ด้วยข้อมูลคุณภาพสูง (Critical Role) จะช่วยยกระดับคุณภาพการบรรยายและสไตล์การเล่นให้ใกล้เคียงมนุษย์
- 4) Prompt Constraints: ระบบถูกควบคุมอย่างเข้มงวดด้วย "Huge System Prompt" เพื่อกำกับพฤติกรรมและขอบเขตการตอบสนองของโมเดล

3. การเปรียบเทียบ (Their Comparison)

- 1) Benchmark: งานนี้เปรียบเทียบ (head-to-head) ประสิทธิภาพระหว่าง AI DM vs. Human DM
- 2) Method: ผู้เข้าร่วมทดลองถูกแบ่งเป็น 2 กลุ่ม (กลุ่มเล่นกับ AI และกลุ่ม Control ที่เล่นกับมนุษย์) โดยเล่นใน Scenario เดียวกัน ("Sunless Citadel"). จากนั้นนำคะแนนความพึงพอใจ (Player satisfaction scores) มาเปรียบเทียบกัน

4. ผลลัพธ์การทดลอง (Their Performance)

ผลลัพธ์ (Performance) ออกมา "ผสมผสาน" (Mixed) โดยใช้การวัดผลด้วย Game Experience Questionnaire (GEQ) และการสัมภาษณ์

- 1) Positive: AI ทำได้ "ใกล้เคียงมนุษย์" ในหลายด้าน และทำได้ ดีกว่ามนุษย์ (อย่างมีนัยสำคัญทางสถิติ $P < 0.05$) ในการสร้าง "Sensory and Imaginative Immersion" (การบรรยายฉากที่สมจริง) (คะแนนเฉลี่ย 4.13/5 เทียบกับมนุษย์ 3.35)
- 2) Negative: มนุษย์ยังคงมี "นัยสำคัญทางสถิติ" ($P < 0.05$) ในด้าน Competence (ความสามารถโดยรวม) และ Flow (ความลื่นไหลของเนื้อเรื่อง)
- 3) AI มีจุดอ่อนร้ายแรง 3 ข้อ (จากการสัมภาษณ์)
 - (1) "Railroading" ผู้เล่น: (3 ใน 7 instances) AI ไม่ยืดหยุ่นและพยายามบังคับผู้เล่นเดินตามเส้นทาง (Struggled with unexpected choices)
 - (2) "Poor Information Delivery": (1 ใน 7 instances) AI ลืมให้ Clue หรือข้อมูลสำคัญในเวลาที่เหมาะสม ทำให้ผู้เล่นสับสน
 - (3) "Lack of Challenge": (3 ใน 7 players) AI สร้างความท้าทายหรือ "ความรู้สึกอันตราย" (Perceived lack of danger) ได้ไม่ดีพอ (เกมง่ายเกินไป)

5. บทเรียนที่นำมาปรับใช้ (What We Will Use)

1) เราจะนำแนวคิดการ "Grounding" AI (การยึดโยงกับข้อมูลจริง) มาปรับใช้ โดยในโครงการนี้จะใช้ Lite 5e Rulebook เป็นฐานข้อมูลอ้างอิง

2) เรานำปัญหา "Railroading" ที่เขาพบ มาเป็น "ปัญหาหลัก" (Key Problem) ที่เราต้องออกแบบสถาปัตยกรรมเพื่อแก้ไข

6. ช่องว่างและข้อจำกัดของงานวิจัย (The Gap/Limit)

1) Complexity Gap: ระบบของ Sakellaridis ใช้วิธีการ Fine-tuning ซึ่งมีความซับซ้อนสูงและไม่เอื้อต่อการนำไปใช้งานจริงแบบพกพา (Not Portable) สำหรับผู้เล่นทั่วไป (Casual User)

2) Our Solution: สถาปัตยกรรม "Two-Path" (โดยเฉพาะ Path B: Creative Actions) ของเราจะเข้ามาเติมเต็มช่องว่างนี้ โดยแก้ปัญหา "Railroading" ด้วยวิธีการที่ต่างออกไป คือการใช้ LLM (ในฐานะ DM) เป็นผู้ตัดสินใจ (Arbitrate) ในสถานการณ์นอกกรอบ และใช้ Rules Engine จัดการกลไกกลุ่ม ซึ่งจะมอบความยืดหยุ่น (Flexibility) ได้ดีกว่าโดยไม่ต้องพึ่งพาการ Fine-tune ที่ซับซ้อน ทำให้ระบบของเราติดตั้งง่ายกว่า (Simpler to Deploy) และมีความยืดหยุ่นสูงกว่า (Less Rigid)

7. แนวทางการประเมินผล (Their Evaluation)

1) เราจะนำหมวดหมู่การวัดผล (Evaluation Categories) ของงานวิจัยนี้มาปรับใช้โดยตรง ได้แก่ Immersion, Narrative Flow, DM Competence (Rule Adherence), และ Challenge Level

2) คำถามในการทดสอบ Pilot Test (Q1-Q6) ในหัวข้อ 3.5 ของโครงการนี้ ได้รับแรงบันดาลใจมาจากตัวชี้วัดเหล่านี้ เพื่อตรวจสอบว่าสถาปัตยกรรม "Two-Path" สามารถแก้ไขปัญหาที่ Sakellaridis พบ (เช่น ปัญหา Railroading ใน Q5 และ Challenge ใน Q1) ได้จริงหรือไม่

2.2.3 Jørgensen et al. (2024) - สถาปัตยกรรมแบบ Multi-Agent

งานวิจัยของ Jørgensen และคณะ (2024) ได้นำเสนอระบบ "ChatRPG" ซึ่งเป็นแนวทางที่น่าสนใจในการออกแบบ AI Dungeon Master ด้วยสถาปัตยกรรมแบบหลายเอเจนต์ (Multi-Agent Architecture) โดยประยุกต์ใช้กรอบแนวคิด ReAct (Reason + Act) เพื่อให้ AI มีกระบวนการ "คิด" (Reasoning) และ "กระทำ" (Action) ที่เป็นขั้นตอน ซึ่งแนวทางนี้ถือเป็นแรงบันดาลใจสำคัญสำหรับการออกแบบเชิงโมดูล (Modular Design) ของโครงการนี้

เพื่อทำความเข้าใจบริบทของงานวิจัยนี้อย่างลึกซึ้ง (Deep Dive) และเพื่อสนับสนุนการออกแบบสถาปัตยกรรมของโครงการนี้ สามารถวิเคราะห์องค์ประกอบของงานวิจัยดังกล่าวได้ดังนี้

1. สถาปัตยกรรมและกติกาที่ใช้ (Their Setting)

1) Architecture: ระบบถูกออกแบบเป็น Multi-agent System ที่ขับเคลื่อนด้วย OpenAI's GPT-4 ภายใต้กรอบการทำงาน ReAct Framework

2) Separation of Concerns: พวกเขา "แยกส่วน" (split) DM ออกเป็นเอเจนต์เฉพาะทาง

(1) "Narrator agent" (ทำหน้าที่เล่าเรื่อง)

(2) "Archivist agent" (ทำหน้าที่จำ State/กฎ)

3) ระบบมีการใช้ Rules Logic Layer หรือเครื่องมือ (Tools/Functions) สำหรับจัดการกลไกเกมที่ซับซ้อน เช่น ฟังก์ชันการต่อสู้ (Battle()) หรือการรักษา (HealCharacter()) ร่วมกับการใช้ฐานข้อมูล (Entity Framework) ในการจดจำสถานะ

2. ข้อสันนิษฐานและข้อจำกัด (Their Assumption)

1) Solo Player Focus: งานวิจัยนี้ตั้งอยู่บนสมมติฐานการเล่นแบบผู้เล่นเดี่ยว (Single-player)

2) Hybrid Superiority: มีข้อสันนิษฐานว่าการ "แยกหน้าที่" (Splitting DM Duties) ระหว่างการเล่าเรื่องและการคุมกฎ จะให้ผลลัพธ์ที่ดีกว่าการใช้ LLM ตัวเดียวทำทุกอย่าง (Monolithic Approach)

3) Stateless Handling: เนื่องจาก LLM มีลักษณะ Stateless (ไม่จดจำบริบทข้ามรอบ) ระบบจึงจำเป็นต้องมี Archivist Agent คอยป้อนข้อมูลความทรงจำ (Memory Injection) จากฐานข้อมูลกลับเข้าไปในการประมวลผลทุกครั้ง

3. การเปรียบเทียบ (Their Comparison)

1) Benchmark: การศึกษานี้เน้นการเปรียบเทียบระหว่าง AI vs. AI เพื่อวัดประสิทธิภาพของสถาปัตยกรรม

2) Method: เปรียบเทียบประสิทธิภาพระหว่าง ระบบ v2 (Agentic, Multi-agent) ซึ่งมีความซับซ้อนสูง กับ ระบบ v1 (Static, Single-prompt) ที่ใช้เพียง Prompt เดียวแบบดั้งเดิม เพื่อแยกแยะผลกระทบและข้อดีของการใช้สถาปัตยกรรมแบบหลายเอเจนต์

4. ผลลัพธ์การทดลอง (Their Performance)

1) Positive: ผู้เล่น "Strongly preferred v2" (ชอบ v2 มากกว่าชัดเจน) โดย v2 ได้คะแนนสูงกว่า v1 (อย่างมีนัยสำคัญทางสถิติ $P < 0.05$) ในด้าน

(1) "Perceived Intelligence" (ความฉลาด)

(2) "Narrative Flow" (ความลื่นไหลของเรื่อง)

(3) "Immersion" (ความดื่มด่ำ)

2) Positive: ระบบ v2 นั้น "Robust" (ไม่พังง่าย) และ "Consistent" (จำเก่ง) เพราะ "Archivist" คอยจํารายละเอียดให้

5. บทเรียนที่นำมาปรับใช้ (What We Will Use)

1) งานวิจัยนี้ถือเป็น "ต้นแบบแนวคิดหลัก" (Core Idea) ของโครงการ โดยเรานำหลักการ "แยกส่วนตรรกะออกจากส่วนเนื้อเรื่อง" (Separating Logic from Narrative) มาประยุกต์ใช้โดยตรง

2) สถาปัตยกรรม "Two-Path" ของเรา คือ "Direct Implementation" (การนำมาประยุกต์ใช้โดยตรง) จากแนวคิดนี้

- (1) Rules Engine & State Tracker ของเรา ทำหน้าที่เทียบเท่า "Archivist Agent"
- (2) LLM (as DM/Narrator) ของเรา ทำหน้าที่เทียบเท่า "Narrator Agent"
- (3) "Router" ของเรา ทำหน้าที่เทียบเท่ากระบวนการคิดแบบ "ReAct" เพื่อตัดสินใจเลือกเส้นทางการทำงาน

6. ช่องว่างและข้อจำกัดของงานวิจัย (The Gap/Limit)

1) High Complexity: ระบบของ Jørgensen มีความซับซ้อนสูงและใช้ทรัพยากรมาก (Powerful but Complex) เนื่องจากต้องใช้ GPT-4, ระบบ Multi-agent, และฐานข้อมูลขนาดใหญ่ ซึ่งทำให้ "ไม่สามารถพกพาได้สะดวก" (Not Portable) และ "เข้าถึงยาก" (Not Casual) สำหรับผู้ใช้ทั่วไป

2) Our Solution: โครงการนี้จะเข้ามาเติมเต็มช่องว่างดังกล่าว โดยใช้ตรรกะการแยกส่วนเดียวกัน แต่พัฒนาบนระบบที่ "เรียบง่ายและเบากว่า" (Simpler, Lightweight) โดยใช้กฎ Lite 5c และพัฒนาเป็นแอปพลิเคชันแบบ Self-contained บน Flutter พร้อมเพิ่มฟีเจอร์ UI และ Voice Input ที่ระบบต้นแบบยังขาดไป

7. แนวทางการประเมินผล (Their Evaluation)

1) เราจะนำ เกณฑ์วัดผลประสบการณ์ผู้ใช้ (User Experience Metrics) ของงานวิจัยนี้ ซึ่งอ้างอิงจาก PXI (Player Experience Inventory) มาปรับใช้

2) โดยเฉพาะตัวชี้วัดด้าน Immersion, Flow, และ Perceived Intelligence ซึ่งได้ถูกนำมาแปลงเป็นคำถามสำหรับการทดสอบ Pilot Test (Q3, Q5) ในหัวข้อ 3.5 เพื่อตรวจสอบคุณภาพของประสบการณ์การเล่น

2.2.4 Tool-Assisted AI DM (Song et al., 2024) - การใช้ RAG และ Function Calling

งานวิจัยล่าสุดโดย Song et al. (2024) ได้นำเสนอแนวคิดการพัฒนา AI DM ด้วยสถาปัตยกรรมแบบผสมผสาน (Hybrid Approach) ที่ผสานการทำงานของ LLM เข้ากับเครื่องมือภายนอก (External Tools) ผ่านกลไก Function Calling และ RAG เพื่อแก้ไขปัญหาความไม่แน่นอนของ LLM ผลการทดลองยืนยันว่าสถาปัตยกรรมนี้ช่วยให้ AI มีความสม่ำเสมอในการรักษาสถานะของเกม (State Consistency) และปฏิบัติตามกฎได้อย่างน่าเชื่อถือ (Reliability) สูงกว่าการใช้ LLM เพียงอย่างเดียวอย่างมีนัยสำคัญ

เพื่อทำความเข้าใจบริบทของงานวิจัยนี้อย่างลึกซึ้ง (Deep Dive) และเพื่อสนับสนุนการออกแบบสถาปัตยกรรมของโครงการนี้ สามารถวิเคราะห์องค์ประกอบของงานวิจัยดังกล่าวได้ดังนี้

1. สถาปัตยกรรมและกติกาที่ใช้ (Their Setting)

1) Rulebook/Tech: งานวิจัยนี้ใช้ Setting ของเกม "Jim Henson's Labyrinth: The Adventure Game" (ซึ่งเป็น TTRPG ที่มีกฎง่ายกว่า D&D)

2) Architecture: สถาปัตยกรรม (Technical Setup) เป็นแบบ Hybrid LLM + External Tools โดยใช้ LLM (GPT-4/3.5) ที่มีความสามารถ Function Calling

3) Tools: พวกเขาได้ Implement "เครื่องมือ" (โค้ด) แยกออกมา 2 ประเภทหลัก

(1) Dice Roll Functions (เช่น `activate_test()`)

(2) State Functions (เช่น `add_item()`, `create_npc()`)

4) RAG: มีการใช้เทคนิค Retrieval-Augmented Generation (RAG) ในการดึง "สรุปกฎ" (Rule Summary) และ "สถานะเกมปัจจุบัน" (Game State) เข้าไปใน Prompt เพื่อให้ AI รับรู้บริบทที่ถูกต้องเสมอ

2. ข้อสันนิษฐานและข้อจำกัด (Their Assumption)

1) Tools Mitigate Limitations: งานวิจัยตั้งสมมติฐานว่า ข้อจำกัดของ LLM ด้านความจำและความแม่นยำ สามารถแก้ไขได้ด้วยการ "มอบเครื่องมือ" (Giving the model tools) ให้ใช้งาน

2) LLM as Decider: ระบบถูกออกแบบโดยสันนิษฐานว่า LLM สามารถ "ตัดสินใจ" (Determine) ได้เอง ว่าสถานการณ์ใดควร "บรรยาย" (Narrate) และสถานการณ์ใดควร "เรียกใช้ฟังก์ชัน" (Call Function)

3) Constraint: การทดลองถูกจำกัดอยู่ภายใต้เนื้อเรื่องแบบสำเร็จรูป (Pre-designed Adventure)

3. การเปรียบเทียบ (Their Comparison)

1) Benchmark: เปรียบเทียบ AI กับ AI โดยแบ่งเป็น 6 Settings (FG-all, FG-dice, FG-states, FG-default, FG-gen, DG) เพื่อทดสอบว่าการ "เปิด/ปิด" Function Calling แต่ละแบบ (เช่น มีทุกอย่าง vs มีแค่ทอยเต๋า vs ไม่มีเลย) ส่งผลต่างกันอย่างไร

2) Method

(1) Human Evaluation: ใช้ผู้ประเมิน 7 คน มา "อ่าน Transcripts" (ไม่ได้เล่นเอง) แล้วให้คะแนน (Likert scale) 3 ด้าน: Consistency (ความต่อเนื่อง/จำเก่ง), Reliability (ความน่าเชื่อถือ/คุมกฎ), และ Interest (ความน่าสนใจ)

(2) Automated Unit Tests: สร้าง Unit Test 30 ข้อ เพื่อวัดความถูกต้องของ State Update (เช่น สั่งให้ AI เพิ่มไอเทม แล้วดูว่ามันเรียก `add_object` หรือไม่)

4. ผลลัพธ์การทดลอง (Their Performance)

- 1) Positive: ผลลัพธ์ชี้ชัดว่าระบบที่ใช้ Function Calling "ชนะขาด" ในด้านคุณภาพและความสม่ำเสมอ (Quality & Consistency) โดย AI สามารถจดจำไอเทมและปฏิบัติตามกฎได้แม่นยำกว่ามาก
- 2) Negative: อย่างไรก็ตาม พบปัญหาสำคัญคือ "การเรียกใช้ฟังก์ชันพรั่ำเพรื่อ" (Overusing Functions) โดย AI บางครั้งอาจเรียกใช้การทอยลูกเต๋าในสถานการณ์ที่ไม่จำเป็น หรือยอมทำตามคำสั่งที่ไร้เหตุผลของผู้เล่นเพียงเพราะมีฟังก์ชันรองรับ

5. บทเรียนที่นำมาปรับใช้ (What We Will Use)

งานวิจัยนี้ "Validate" (พิสูจน์ยืนยัน) สถาปัตยกรรม Path A ของเราอย่างชัดเจน

- 1) Rules Engine ของเรา ก็คือ External Tool / Function Calling ของเขา
- 2) เราจะ "Adopt" (นำมาใช้) แนวคิดของพวกเขาในการ "Encoding game mechanics as callable functions" (เช่น roll_dice, apply_damage) เพื่อรับประกันความแม่นยำของกฎ

6. ช่องว่างและข้อจำกัดของงานวิจัย (The Gap/Limit)

- 1) Gap 1 (Offline vs. Live): ระบบของเขาประเมินผลแบบ "Offline" (อ่าน Transcript) ไม่ใช่ "Live Play" (ผู้เล่นจริง) งานเรอดูช่องว่างนี้ด้วย Pilot Test (User-focused กว่า)
- 2) เราจะ "Adopt" (นำมาใช้) แนวคิดของพวกเขาในการ "Encoding game mechanics as callable functions" (เช่น roll_dice, apply_damage) เพื่อรับประกันความแม่นยำของ Gap 2 (Overusing Tools): ปัญหาใหญ่ของเขาคือ LLM "ใช้เครื่องมือพรั่ำเพรื่อ" (Overusing functions) เพราะ LLM ต้อง "ตัดสินใจเอง"
- 3) Our Solution : "Two-Path Router" ของเรา แก้ปัญหานี้ได้ เพราะ Router ของเรา รู้กฎ Lite 5e และ "บังคับ" (Forces) เฉพาะ Action ไหนเป็น "Fixed" (Path A) (ต้องเรียก Rules Engine) และ Action ไหนเป็น "Creative" (Path B) (ให้ LLM คิด) ทำให้ระบบเรา "Reliable" (น่าเชื่อถือ) กว่า และ "Casual-friendly" (ไม่น่ารำคาญ) มากกว่า

7. แนวทางการประเมินผล (Their Evaluation)

- 1) เราจะนำแนวคิด "Automated Unit Tests" มาใช้โดยตรง
- 2) "Functional Testing" (หัวข้อ 3.5) ของเรา คือการนำแนวคิดนี้มาปฏิบัติจริง โดยการสร้าง Scripted Scenarios เพื่อทดสอบการทำงานของ Rules Engine ซึ่งเป็นคำตอบที่ชัดเจนสำหรับคำถามที่ว่า "จะตรวจสอบความถูกต้องของกฎได้อย่างไร"

2.2.5 สรุปและบทเรียนที่ได้รับจากงานวิจัยที่เกี่ยวข้อง

จากการทบทวนและวิเคราะห์งานวิจัยที่เกี่ยวข้อง (Section 2.2.1 - 2.2.4) สามารถสรุปบทเรียนสำคัญ (Key Takeaways) ซึ่งนำไปสู่การออกแบบสถาปัตยกรรมของโครงการนี้ได้ดังนี้

1. ปัญหาที่พิสูจน์แล้ว: สถาปัตยกรรมแบบ Monolithic (LLM-only) ไม่เพียงพอ งานวิจัยของ Triyason (2023) และ Sakellariadis (2024) ได้แสดงให้เห็นอย่างชัดเจนถึงข้อจำกัดของสถาปัตยกรรมที่ใช้ LLM เพียงอย่างเดียว (Monolithic Architecture) แม้ว่า LLM จะมีความเป็นเลิศในด้านการสร้างสรรค์เนื้อหา และการบรรยาย (Narrative Quality) แต่กลับประสบความล้มเหลวในภารกิจที่ต้องการความแม่นยำเชิงตรรกะ (Deterministic Tasks) ได้แก่

- 1) การไม่ปฏิบัติตามกฎ (Poor Rule Adherence): ไม่สามารถคำนวณกลไกเกมที่ซับซ้อนได้อย่างถูกต้องแม่นยำ (Hallucination)
- 2) การไม่สามารถรักษาสถานะ (State Inconsistency): ขาดหน่วยความจำระยะยาวที่เชื่อถือได้ ทำให้ลืมสถานะสำคัญของเกม (Statelessness)
- 3) การขาดความยืดหยุ่น (Railroading): มักจะบังคับให้ผู้เล่นดำเนินตามเส้นเรื่องที่กำหนดไว้ล่วงหน้าเมื่อเผชิญกับการกระทำที่ซับซ้อน

2. ทางออกที่ได้รับการยอมรับ: สถาปัตยกรรมแบบผสมผสาน (Hybrid/Tools) แนวทางที่ได้รับการพิสูจน์แล้วว่าประสบความสำเร็จมากกว่า คือการใช้สถาปัตยกรรมแบบผสมผสาน (Hybrid Architecture) หรือแบบแยกส่วน (Decomposed) ดังที่ปรากฏในงานวิจัยของ Jørgensen et al. (2024) และกลุ่ม Tool-Assisted (2024) ซึ่งชี้ให้เห็นว่า กุญแจสำคัญคือการ "แยกส่วนการทำงาน" (Separation of Concerns) อย่างชัดเจน

- 1) แยกส่วนการเล่าเรื่อง (Narrative): ให้ LLM รับผิดชอบในสิ่งที่ถนัด คือการสร้างสรรค์บทบรรยายและบทสนทนา
- 2) แยกส่วนตรรกะและสถานะ (Logic & State): ให้ระบบเชิงสัญลักษณ์ (Symbolic System) หรือโค้ดโปรแกรม รับผิดชอบในการคำนวณและบันทึกสถานะ
- 3) การใช้เครื่องมือ (Tool Use): เชื่อมต่อทั้งสองส่วนด้วยกลไก Function Calling เพื่อให้ LLM สามารถเรียกใช้โค้ดในการจัดการงานที่ต้องการความแม่นยำ

3. บทสรุปและการนำมาปรับใช้: สถาปัตยกรรม "Two-Path" (Hybrid-Arbitrated) จากบทเรียนข้างต้น โครงการนี้จึงเลือกใช้สถาปัตยกรรมแบบ "Two-Path" (Hybrid-Arbitrated) ซึ่งถือเป็นทางเลือกที่สมเหตุสมผล (Reasonable) ที่สุด โดยเป็นการนำหลักการ Hybrid มาปรับใช้ให้เหมาะสมกับขอบเขตของ Lite 5c (ภาคผนวก ก) ดังนี้

1) Path A (Rule-First): สำหรับการกระทำที่มีกฎตายตัว (Fixed Actions) เช่น การโจมตี ระบบจะนำแนวคิด Function Calling มาใช้ โดยให้ Rules Engine ทำหน้าที่ประมวลผลก่อนเพื่อรับประกันความถูกต้อง 100%

2) Path B (LLM-First): สำหรับการกระทำเชิงสร้างสรรค์ (Creative Actions) เช่น การสำรวจ ระบบจะรักษาจุดแข็งด้านความคิดสร้างสรรค์ของ LLM ไว้ โดยให้ LLM (ในฐานะ DM) ทำหน้าที่เป็นผู้ตัดสิน (Arbitrator) ในการกำหนดค่าความยาก (DC) ก่อนส่งต่อให้ระบบคำนวณ

4. การเติมเต็มช่องว่างของงานวิจัย (The Gap): ความสามารถในการพกพา (Portability) งานวิจัยส่วนใหญ่ที่นำเสนอสถาปัตยกรรมแบบ Hybrid มักมีความซับซ้อนสูง (High Complexity) และต้องใช้ทรัพยากรประมวลผลจำนวนมาก (เช่น Multi-agent systems ที่ซับซ้อน) หรือใช้กฎ D&D 5e เต็มรูปแบบที่ยุ่งยาก โครงการนี้จึงมุ่งเน้นที่จะเติมเต็มช่องว่างดังกล่าว โดยการประยุกต์ใช้สถาปัตยกรรม Hybrid ที่มีประสิทธิภาพ ควบคู่ไปกับการ "ลดขอบเขต" (Scope Reduction) ด้วยชุดกติกา Lite 5e เพื่อสร้างระบบ AI Dungeon Master ที่มีทั้งความน่าเชื่อถือ (Reliable) และที่สำคัญคือมีความ "พกพาง่าย" (Portable) และ "เข้าถึงง่าย" (Casual) สำหรับผู้เล่นทั่วไป ซึ่งเป็นจุดที่งานวิจัยก่อนหน้านี้ยังไม่ได้มุ่งเน้น

ลำดับ	แผนดำเนินงาน	2568				2569				
		ก.ย.	ต.ค.	พ.ย.	ธ.ค.	ม.ค.	ก.พ.	มี.ค.	เม.ย	พ.ค.
6.	บูรณาการระบบเป็น Prototype-B (ผนวก ฟังก์ชันเสริม ASR/TTS) และ ทดสอบเบื้องต้น									
7.	ทดลองใช้งานระบบ (Pilot Test) และ รวบรวมผลตอบรับ									
8.	ปรับปรุงระบบขั้นสุดท้าย และจัดทำ รายงานฉบับสมบูรณ์									

ตารางที่ X แผนภูมิแกนต์

3.2 เครื่องมือและเทคโนโลยีที่ใช้

การพัฒนาโครงการนี้อาศัยเครื่องมือซอฟต์แวร์และเทคโนโลยีที่หลากหลาย ซึ่งถูกคัดเลือกมาโดยคำนึงถึงเป้าหมายหลักคือการสร้างแอปพลิเคชันที่ทำงานได้ด้วยตนเองบนหลายแพลตฟอร์ม (Cross-platform) และง่ายต่อการนำไปใช้งาน

1. ภาษาโปรแกรมและเฟรมเวิร์ก Dart เป็นภาษาหลักในการพัฒนา โดยใช้เฟรมเวิร์ก Flutter ในการสร้างแอปพลิเคชันส่วนหน้า (Frontend) เหตุผลที่เลือกใช้ Flutter เนื่องจากมีจุดแข็งในการใช้ฐานโค้ดเดียว (Single Codebase) แต่สามารถคอมไพล์เป็นแอปพลิเคชัน ที่ทำงานได้บนหลายระบบปฏิบัติการ เช่น Windows, macOS, Android และ iOS ซึ่งตอบโจทย์เป้าหมายของโครงการที่ต้องการให้ผู้ใช้สามารถเข้าถึงได้ง่าย
2. ส่วนประมวลผลหลัก (Backend Logic) โลจิกหลักของเกม เช่น Rules Engine และ State Tracker จะถูกพัฒนาด้วยภาษา Dart เช่นกัน เพื่อให้เป็นส่วนหนึ่งของแอปพลิเคชัน Flutter โดยตรง แนวทางนี้ช่วยลดความซับซ้อนของสถาปัตยกรรมลงอย่างมาก โดยไม่จำเป็นต้องมีการตั้งเซิร์ฟเวอร์แยกต่างหาก และทำให้แอปพลิเคชันสามารถทำงานได้อย่างสมบูรณ์ในตัวเอง (Self-contained)
3. การรู้จำเสียงพูด (ASR) (ฟังก์ชันเสริม) ใช้โมเดลโอเพนซอร์สสำเร็จรูปที่สามารถทำงานบนอุปกรณ์ได้โดยตรง (On-device) โดยจะมุ่งเน้นการบูรณาการไลบรารีเข้ากับ Flutter เพื่อเป็นทางเลือก (Alternative Input) ในการป้อนข้อมูล นอกเหนือจากการพิมพ์ และเพื่อรักษาความเป็นส่วนตัวของข้อมูลเสียง

4. การแปลงข้อความเป็นเสียง (Text-to-Speech - TTS) จะมีการเพิ่มฟังก์ชันเสริมในการอ่านออกเสียงข้อความบรรยายจาก AI DM ด้วยเสียงสังเคราะห์แบบพื้นฐาน โดยอาศัยไลบรารีมาตรฐานที่มีใน Flutter เพื่อเพิ่มทางเลือกในการรับรู้ข้อมูลของผู้เล่นโดยไม่เพิ่มความซับซ้อนให้กับระบบมากนัก
5. แบบจำลองภาษาขนาดใหญ่ (LLM) เรียกใช้งานผ่าน REST API ไปยังผู้ให้บริการภายนอก โดยจะเลือกใช้โมเดลที่มีค่าใช้จ่ายไม่สูง เพื่อให้สอดคล้องกับความต้องการที่ ต้องการระบบที่เข้าถึงง่ายและรวดเร็ว คณะผู้จัดทำจึงได้ทำการวิเคราะห์เปรียบเทียบ API ของโมเดลต่างๆ ดังรายละเอียดในหัวข้อ 3.2.1

3.2.1 การวิเคราะห์เปรียบเทียบและเหตุผลในการเลือก LLM API (Comparative Analysis for LLM Selection)

จากการทบทวนวรรณกรรมในบทที่ 2 พบว่างานวิจัยส่วนใหญ่ที่เน้นประสิทธิภาพสูงสุด (High Performance) มักเลือกใช้โมเดลขนาดใหญ่ที่มีความซับซ้อนและราคาสูง (เช่น GPT-4) อย่างไรก็ตามเนื่องจากโครงการนี้มีเป้าหมายหลักคือความเป็น "Casual" (ต้นทุนต่ำ) และ "Portable" (ตอบสนองรวดเร็ว) ดังนั้น เกณฑ์ในการคัดเลือก (Criteria) จึงให้ความสำคัญกับ ต้นทุน (Cost) และ ความเร็ว (Speed/Latency) เป็นหลัก โดยยังคงต้องรองรับฟีเจอร์สำคัญคือ Function Calling และ Context Window ขนาดใหญ่เพื่อรองรับการเล่นระยะยาว

เกณฑ์ (Criteria)	Gemini 2.5 Flash-Lite (ตัวเลือกของเรา)	GPT-5 nano (คู่แข่งสาย ประหยัด)	GPT-5 mini (คู่แข่ง สายกลาง)	Claude 3 Haiku 4.5 (คู่แข่งสายเร็ว)
1. Cost (ราคา/Token) (Input/Output ต่อ 1M) (Goal: "Casual" ต้องถูก)	\$0.10 / \$0.40	\$0.05 / \$0.40	\$0.25 / \$2.00	\$1.00 / \$5.00
2. Speed (ความเร็ว/ Latency) (Goal: "Portable" ต้องเร็ว)	เร็วที่สุด (Ultra Fast)	เร็วมาก (Very fast)	เร็ว (Fast)	เร็วที่สุด (Fastest)
3. Context Window (หน่วยความจำ) (Goal: D&D เรื่องยาว)	1 ล้าน Tokens	400K Tokens	400K Tokens	(ไม่ระบุชัด แต่ Sonnet/Opus มี 1M+)
4. Function Calling (Goal: รองรับ "Two-Path")	รองรับ (Supported)	รองรับ (Supported)	รองรับ (Supported)	รองรับ (Supported)

เกณฑ์ (Criteria)	Gemini 2.5 Flash-Lite (ตัวเลือกของเรา)	GPT-5 nano (คู่แข่งสาย ประหยัด)	GPT-5 mini (คู่แข่ง สายกลาง)	Claude 3 Haiku 4.5 (คู่แข่งสายเร็ว)
5. Quality (คุณภาพ) (Goal: ดีพอสำหรับ Lite 5e)	ดี (Optimized for cost-efficiency)	พอใช้ (Fastest, cheapest... for summarization)	ดีมาก (Great for well-defined tasks)	ดีมาก (Fastest, cost-efficient)

ตารางที่ X เปรียบเทียบคุณสมบัติของ LLM API รุ่นประหยัดที่มีศักยภาพ ณ ไตรมาสที่ 4 ปี 2025

บทสรุปการคัดเลือก (Selection Justification)

- 1. ความสมดุลระหว่างราคาและประสิทธิภาพ (Cost-Performance Balance): แม้ GPT-5 Nano จะมีราคา Input ที่ต่ำกว่าเล็กน้อย แต่ Gemini 2.5 Flash-Lite มีจุดเด่นที่เหนือกว่าอย่างชัดเจนในเรื่องของ Context Window ที่รองรับได้ถึง 1 ล้าน Tokens (เทียบกับ 400K ของคู่แข่ง) ซึ่งเป็นปัจจัยสำคัญอย่างยิ่งสำหรับเกม Tabletop RPG ที่ต้องจดจำเนื้อเรื่องและสถานะของตัวละครในระยะยาว เพื่อป้องกันปัญหาการลืมบริบท (Statelessness)
- 2. ความเร็วในการตอบสนอง (Latency): โมเดลนี้ถูกออกแบบมาเพื่อ High Throughput และ Low Latency ซึ่งเหมาะสมที่สุดสำหรับการสร้างประสบการณ์การเล่นที่ลื่นไหล (Flow Management) บนอุปกรณ์พกพา
- 3. ความเหมาะสมของฟีเจอร์: รองรับ Function Calling อย่างสมบูรณ์ ซึ่งเป็นกลไกหลักในการเชื่อมต่อกับ Rules Engine ในสถาปัตยกรรมแบบ Two-Path ของเรา

สรุปการเลือก Gemini 2.5 Flash-Lite จึงเป็นทางเลือกที่สมเหตุสมผลที่สุด (Most Reasonable Choice) เพราะมีความ "สมดุล" (Balance) ระหว่าง ราคา (Cost), ความเร็ว (Speed), และ ฟีเจอร์ (1M Context + Function Calling) ซึ่งตอบโจทย์หลักของโครงการในเรื่อง "Portable" (พกพาง่าย) และ "Casual" (เข้าถึงง่าย/ต้นทุนต่ำ) ได้อย่างลงตัวที่สุด

3.3 การออกแบบและพัฒนาระบบ

หัวใจของระเบียบวิธีวิจัยนี้คือการออกแบบสถาปัตยกรรมซอฟต์แวร์ที่สามารถบูรณาการเทคโนโลยีต่างๆ เข้าด้วยกันได้อย่างมีประสิทธิภาพ โดยทุกการออกแบบ ได้รับการสนับสนุนโดย Problem Statement (บทที่ 1) และ Literature Review (บทที่ 2) เพื่อให้มั่นใจว่าสถาปัตยกรรมที่ได้นั้น สมเหตุสมผล

3.3.1 ภาพรวมสถาปัตยกรรมระบบ (System Architecture Overview)

ระบบ AI Dungeon Master ถูกออกแบบตามสถาปัตยกรรมแบบ "Two-Path Architecture" (Hybrid-Arbitrated) ซึ่งเป็นการปรับปรุงแนวคิด Hybrid AI เพื่อให้เหมาะสมกับการเล่น Tabletop RPG ที่ต้องการทั้งความแม่นยำของกฎ (Rule Adherence) และความยืดหยุ่นในการเล่าเรื่อง (Narrative Flexibility) สถาปัตยกรรมนี้ยึดหลักการ "Stateless Machine" และ "Single Source of Truth" โดยใช้ GameState เป็นแหล่งข้อมูลความจริงเพียงหนึ่งเดียว และใช้ "Router" เป็นกลไกหลักในการตัดสินใจเลือกเส้นทางการประมวลผล โดยอ้างอิงจาก ชุดกติกา Lite 5e (ภาคผนวก ก) ดังนี้

1. ปรัชญาการออกแบบ (Core Philosophy)

1) Stateless Machine: Rules Engine ถูกออกแบบให้รับข้อมูลนำเข้า (Input) เพื่อทำการคำนวณและส่งผลลัพธ์ (Output) ออกไปเท่านั้น โดยไม่มีการจดจำสถานะค้างไว้ เพื่อลดความซับซ้อนและป้องกันข้อผิดพลาด (Side Effects)

2) Single Source of Truth: ข้อมูลสถานะของเกมทั้งหมด (GameState) จะถูกเก็บและจัดการในรูปแบบ Dart Object เพียงจุดเดียว AI จะไม่มีสิทธิ์จดจำหรือเปลี่ยนแปลงสถานะโดยพลการ

3) Deterministic Logic: การคำนวณทางคณิตศาสตร์ทั้งหมด (เช่น การทอยลูกเต๋า, การคำนวณความเสียหาย) จะดำเนินการผ่านโค้ดโปรแกรม 100% เพื่อป้องกันปัญหา Hallucination ของ AI

2. โครงสร้างข้อมูล (Data Models) ระบบจัดการข้อมูลโดยแบ่งเป็น 2 รูปแบบเพื่อประสิทธิภาพสูงสุด

1) Internal (ภายในแอปพลิเคชัน): ใช้ Dart Class เพื่อความรวดเร็วในการประมวลผลและการแสดงผล UI

(1) Character Model: เก็บค่า ID, Name, Role, Stats (PHYS/MENT/SOC), HP, AC, Zone และ Inventory

(2) Game State Model: เก็บรายการผู้เล่น (Players), ศัตรู (Enemies), รอบการเล่น (Turn Count), และประวัติการกระทำ (Action Log)

2) External (สำหรับส่ง AI): ใช้รูปแบบ TOON (Token-Oriented Object Notation) ในการแปลงข้อมูลสถานะเกมเพื่อส่งไปยัง LLM ซึ่งช่วยลดปริมาณ Token ได้ถึง 40% และทำให้ AI ประมวลผลโครงสร้างข้อมูลได้แม่นยำขึ้น

3. การไหลของข้อมูลและกระบวนการทำงาน (Data Flow & Execution) กระบวนการทำงานของระบบเริ่มจากการรับอินพุตจากผู้เล่น และผ่านการประมวลผลโดย "Router" ซึ่งทำหน้าที่วิเคราะห์เจตนา (Intent) และเลือกเส้นทางการทำงานที่เหมาะสม ดังนี้

1) ขั้นตอนที่ 1: Input & Routing

(1) ระบบรับคำสั่งจากผู้เล่น (Text/Voice) และส่งไปยัง Router (ซึ่งใช้ Gemini 2.5 Flash-Lite เป็น Classifier)

(2) Router จะจำแนกคำสั่งออกเป็น 2 ประเภท

1. Fixed Action: คำสั่งที่ตรงกับกฎ Lite 5e (เช่น Attack, Cast, Move, Use) → ส่งไป Path A
2. Creative Action: คำสั่งที่ซับซ้อนหรือไม่อยู่ในกฎ (เช่น "I want to tie the rope to trip the goblin") → ส่งไป Path B

2) ขั้นตอนที่ 2: Path Processing

(1) Path A (Rule-First): สำหรับ Fixed Action

1. ส่งข้อมูลเจตนาไปยัง Rules Engine
2. Rules Engine ทำการคำนวณผลลัพธ์ (เช่น ทอย d20 + PHYS เทียบกับ AC) และอัปเดต GameState
3. ระบบสร้าง RAG Context (ประกอบด้วยผลลัพธ์การคำนวณ + สถานะล่าสุด + เนื้อเรื่อง) ส่งให้ LLM
4. LLM ทำหน้าที่เป็น Narrator บรรยายผลลัพธ์ที่เกิดขึ้นจริง (เช่น "Your sword strikes true! Dealing 6 damage to the goblin.")

(2) Path B (LLM-First): สำหรับ Creative Action

1. ระบบส่ง RAG Context (สถานะ + กฎ 3 Stats) ให้ LLM
2. LLM ทำหน้าที่เป็น Arbitrator วิเคราะห์เจตนาและกำหนดเงื่อนไขการทอย (เช่น "ต้องทอย MENT Check ที่ความยาก DC 12")
3. ส่งเงื่อนไขกลับมาให้ Rules Engine ทำการคำนวณผล (Success/Fail)
4. LLM รับผลการคำนวณและบรรยายสถานการณ์ต่อตามผลลัพธ์ (e.g., "You successfully decipher the ancient text...")

3) ขั้นตอนที่ 3: Combat Loop Management

(1) เพื่อแก้ปัญหาการใช้งานบนอุปกรณ์เดียว ระบบใช้กลไก "Side Initiative" โดยแบ่งรอบการเล่นเป็น 2 เฟส

1. Phase 1 (Players Turn): เปิดโอกาสให้ผู้เล่นทุกคนปรึกษาและเลือกกระทำได้อย่างอิสระ

2. Phase 2 (Enemy Turn): AI จะคำนวณการกระทำของศัตรูทั้งหมดในรอบเดียวและบรรยายสรุปเหตุการณ์ เพื่อความรวดเร็ว

4) ขั้นตอนที่ 4: Output

(1) ระบบนำผลลัพธ์ทั้งหมด (Narrative text, Dice results, Updated state) มาแสดงผลบน UI (Map, Log, Stats) ให้ผู้เล่นรับทราบ

(หมายเหตุ: แผนภาพ Architecture Diagram จะถูกจัดทำเพิ่มเติมเพื่อแสดงรายละเอียดการไหลของข้อมูลในรายงานฉบับสมบูรณ์)

3.3.2 การพัฒนาส่วนประกอบหลัก (Development of Key Modules)

เพื่อให้การพัฒนาเป็นไปอย่างมีประสิทธิภาพและสอดคล้องกับสถาปัตยกรรมที่ออกแบบไว้ คณะผู้จัดทำได้แบ่งส่วนประกอบหลักออกเป็น 3 ส่วน โดยมีการกำหนดโครงสร้างไฟล์ (File Structure) และแนวทางการพัฒนาเชิงเทคนิคที่ชัดเจน ดังนี้

1. Rules Engine & Game State Tracker (ระบบคำนวณกฎและติดตามสถานะ) ส่วนนี้ถือเป็นหัวใจสำคัญของความถูกต้อง (Rule Adherence) โดยพัฒนาเป็น Pure Dart Class ที่ทำงานเป็นเอกเทศจากส่วนแสดงผล (UI) เพื่อให้ง่ายต่อการทดสอบ (Unit Testing) โดยมีโครงสร้างการทำงานดังนี้

1) Stateless Architecture: Rules Engine ถูกออกแบบให้ทำงานแบบ "ไร้สถานะ" (Stateless) โดยรับข้อมูล GameState และคำสั่ง Action เข้ามาประมวลผล แล้วส่งผลลัพธ์กลับออกไปเท่านั้น ไม่มีการเก็บค่าค้างไว้ในตัว Engine เอง เพื่อลดความซับซ้อนและป้องกันบั๊ก (Side Effects)

2) Deterministic Logic: การคำนวณทางคณิตศาสตร์ทั้งหมด (เช่น การทอยลูกเต๋า rollDice, การคำนวณความเสียหาย resolveAttack) จะถูกเขียนเป็นโค้ดโปรแกรมที่ตายตัว 100% โดยไม่อนุญาตให้ AI เป็นผู้คำนวณ

3) Class-based Data: ข้อมูล Character และ GameState ถูกเก็บในรูปแบบ Class Object เพื่อความรวดเร็วในการประมวลผลภายในแอปพลิเคชัน

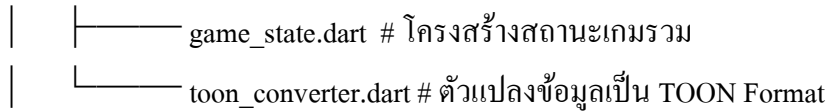
โครงสร้างไฟล์ (Project Structure)

lib/

```

├── logic/           # ส่วนคำนวณกฎ (Stateless)
│   ├── rules_engine.dart # ฟังก์ชันหลัก (resolveAttack, resolveCheck)
│   ├── dice_roller.dart # ฟังก์ชันสุ่มตัวเลข (rollD20, rollDamage)
│   └── abilities.dart   # ตรรกะความสามารถพิเศษ (Second Wind, Smite)
├── models/         # ส่วนเก็บข้อมูล (State)
│   └── character.dart   # โครงสร้างข้อมูลตัวละคร (Stats, HP, Inventory)

```



2. LLM Integration and Prompt Engineering (การเชื่อมต่อและตั้งการ AI) ส่วนนี้เน้นการออกแบบ "Prompt Template" ที่มีประสิทธิภาพเพื่อควบคุมพฤติกรรมของ AI ให้สอดคล้องกับบทบาท Game Master โดยใช้เทคนิคดังนี้

1) Structured Prompting: ใช้การแบ่งโครงสร้างคำสั่งด้วย Tag ที่ชัดเจน (เช่น <context>, <rules>, <history>, <user_action>) เพื่อชี้นำให้ LLM แยกแยะบริบทและสร้างผลลัพธ์ที่แม่นยำ ลดโอกาสการเกิดภาพหลอน (Hallucination)

2) Token Optimization (TOON Format): เพื่อให้สอดคล้องกับเป้าหมายด้าน "ความคุ้มค่า" (Cost-efficiency) และ "ความเร็ว" (Low Latency) คณะผู้จัดทำได้เลือกใช้รูปแบบข้อมูล TOON (Token-Oriented Object Notation) ในการส่งข้อมูลสถานะเกมไปยัง LLM แทนการใช้ JSON แบบดั้งเดิม

(1) เหตุผลสนับสนุน (Technical Justification): จากการศึกษาเปรียบเทียบพบว่า TOON สามารถลดจำนวน Token ได้เฉลี่ย 40-50% เมื่อเทียบกับ JSON เนื่องจากโครงสร้างที่กระชับกว่า (พยาน YAML และ CSV) ซึ่งช่วยลดค่าใช้จ่าย API และลดเวลาในการประมวลผล (Processing Time) ของโมเดลลงได้อย่างมีนัยสำคัญ โดยยังคงความแม่นยำในการอ่านข้อมูล (Retrieval Accuracy) ไว้ได้สูงถึง 74% (เทียบกับ JSON ที่ 70%)

3. User Interface (UI) Development (การพัฒนาส่วนต่อประสานกับผู้ใช้) พัฒนาด้วยเฟรมเวิร์ก Flutter โดยเน้นการออกแบบที่ "Minimalist & Single Device" เพื่อรองรับการเล่นร่วมกันบนอุปกรณ์เดียว

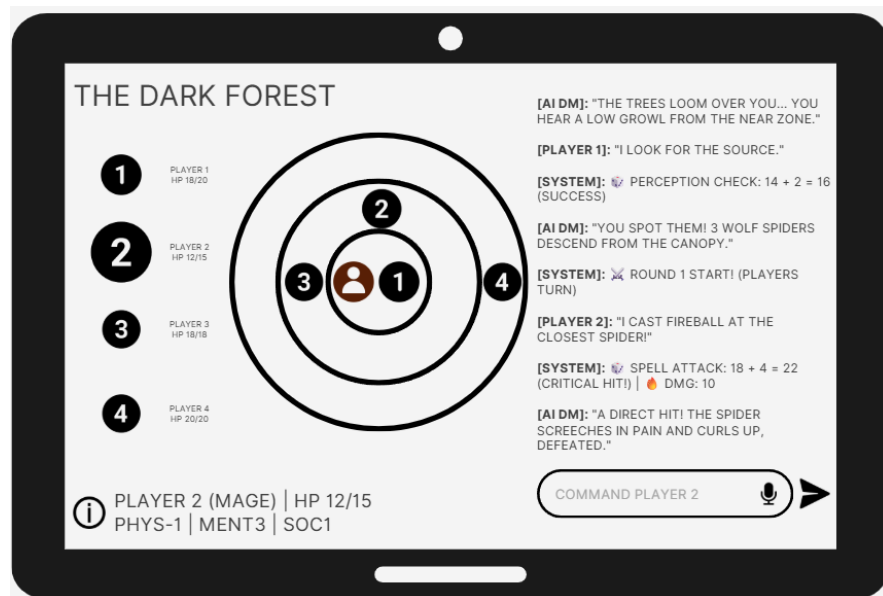
1) Component-based Design: แยกส่วนประกอบ UI ออกเป็น Widget ย่อย (เช่น CharacterCard, ZoneMap, ActionLog) เพื่อให้ง่ายต่อการดูแลรักษาและปรับแก้

2) Real-time Feedback: เชื่อมต่อ UI เข้ากับ GameState เพื่อให้หน้าจอบันทึกค่าพลังชีวิต (HP Bar) และสถานะตัวละครทันทีที่มีการเปลี่ยนแปลงจาก Rules Engine

3.3.3 การออกแบบส่วนต่อประสานกับผู้ใช้ (User Interface Design)

เพื่อให้สอดคล้องกับเป้าหมายการใช้งานแบบ "Single Device" (เล่นบนอุปกรณ์เดียวร่วมกัน) และรองรับกฎ "Lite 5e" คณะผู้จัดทำได้ออกแบบหน้าจอการใช้งาน (User Interface) โดยเน้นความเรียบง่าย (Minimalist) และการเข้าถึงข้อมูลสำคัญได้อย่างรวดเร็ว เพื่อให้ตอบโต้ความต้องการของผู้ใช้ (User Requirements) ดังนี้

1. หน้าจอหลัก (Main Game Screen) หน้าจอหลักถูกออกแบบให้เป็นศูนย์รวมข้อมูล (Dashboard) เพื่อลดความซับซ้อนในการสลับหน้าจอ โดยแบ่งพื้นที่ออกเป็น 3 ส่วนหลัก ได้แก่ (1) ส่วนเลือกตัวละคร (Character Selector), (2) แผนที่โซน (Zone Map), และ (3) บันทึกเหตุการณ์ (Action Log) ดังแสดงในรูปที่ X



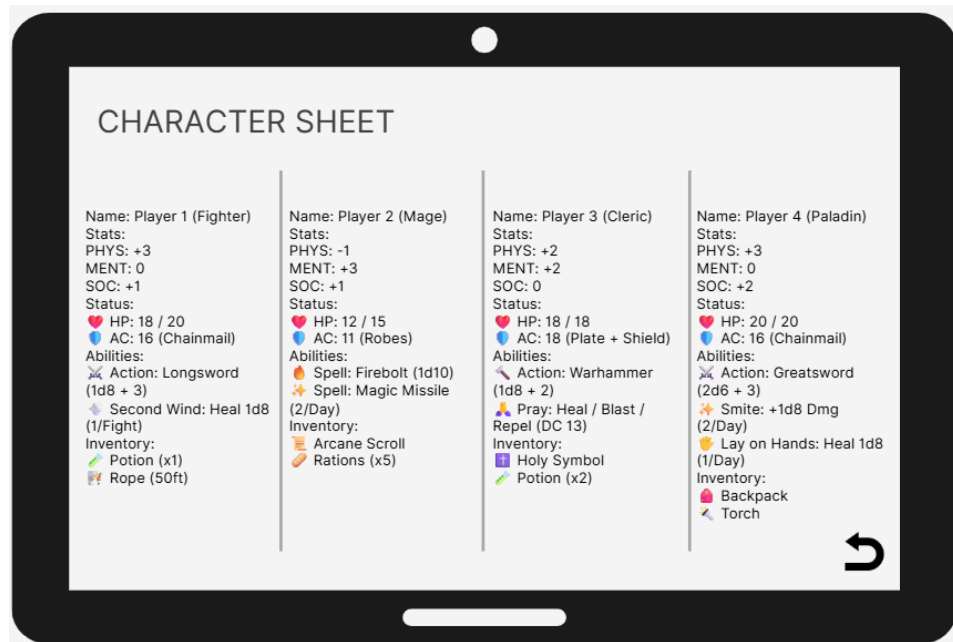
รูปที่ X การออกแบบหน้าจอหลัก (Main Game Screen) ที่แสดงองค์ประกอบสำคัญและการจัดวางแบบ Single Device

1) Zone Map (ตรงกลาง) ใช้การแสดงผลแบบวงกลม 3 ระยะ (Near, Mid, Far) แทนตาราง Grid เพื่อให้เข้าใจง่ายและเหมาะสมกับการเล่นแบบ Casual

2) Character Selector (ด้านซ้าย) แถบเลือกตัวละครช่วยให้ผู้เล่นหลายคนสามารถสลับกันควบคุมตัวละครของตนเองได้สะดวกในอุปกรณ์เดียว

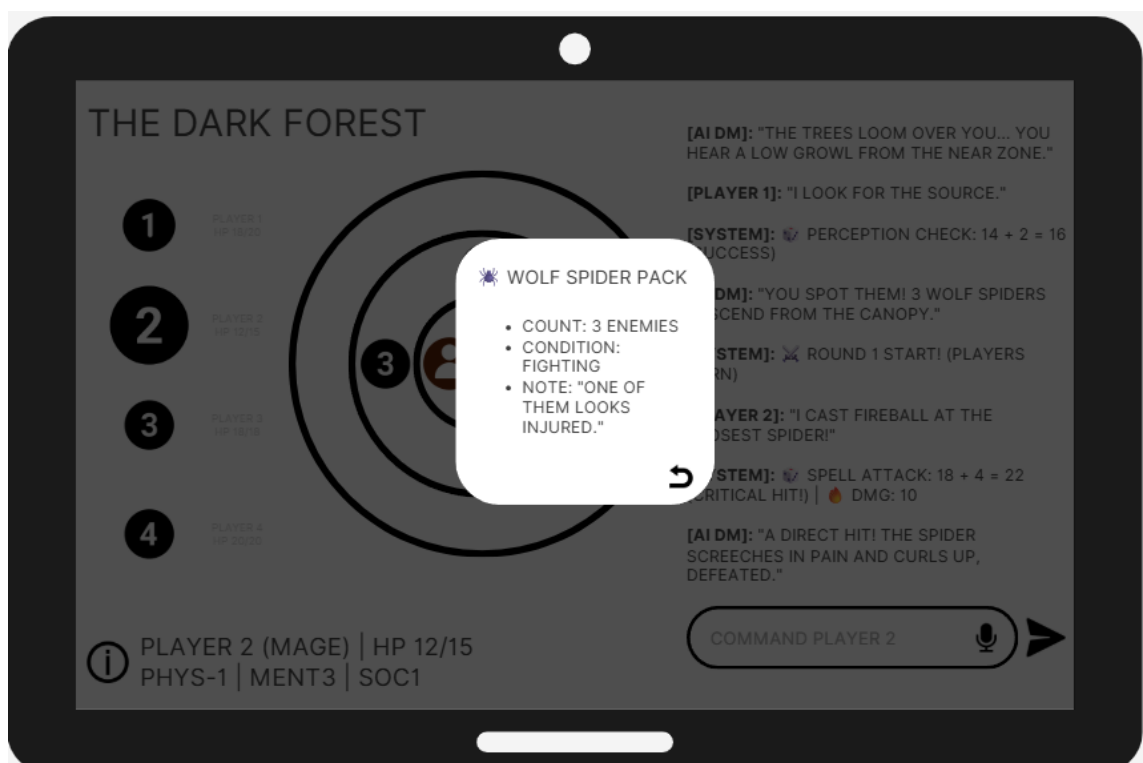
3) Action Log (ด้านขวา) แสดงผลการดำเนินเรื่องและผลลัพธ์ทางกลไก (System Log) แยกจากกันเพื่อความชัดเจน

2. หน้าจอรายละเอียดตัวละคร (Character Sheet Overlay) เมื่อผู้เล่นต้องการตรวจสอบข้อมูลเชิงลึก ระบบจะแสดงหน้าต่างแบบ Overlay ทับหน้าจอหลัก เพื่อให้ผู้เล่นสามารถเปรียบเทียบค่าสถานะ (Stats), พลังชีวิต (HP), และรายการไอเทม (Inventory) ของตัวละครทั้ง 4 ได้พร้อมกัน ดังแสดงในรูปที่ X



รูปที่ X หน้าจอรายละเอียดตัวละคร (Character Sheet) แสดงข้อมูลรวมของปาร์ตี้

3. การแสดงข้อมูลศัตรู (Enemy Inspection) เพื่อลดความซับซ้อนของหน้าจอ ระบบใช้กลไก "Popup" ขนาดเล็กเมื่อกดที่ไอคอนศัตรู เพื่อแสดงข้อมูลสถานะปัจจุบัน (เช่น Healthy, Injured) และจำนวนศัตรูในกลุ่ม โดยไม่บดบังพื้นที่การเล่นหลัก ดังแสดงในรูปที่ X



รูปที่ X หน้าต่างข้อมูลศัตรู (Enemy Inspect Popup)

3.4 แหล่งข้อมูลและความรู้ของระบบ (System Knowledge and Data Sources)

1. ชุดกติกา Lite 5e คณะผู้จัดทำได้ออกแบบและสร้างเอกสารสรุปกฎ Lite 5e ขึ้นมาโดยเฉพาะ ซึ่งจะทำหน้าที่เป็นเอกสารข้อกำหนด (Specification Document) สำหรับการพัฒนาฟังก์ชันทั้งหมดใน Rules Engine
2. เนื้อเรื่องและสถานการณ์ (Scenario Content): สำหรับต้นแบบ (Prototype) ระบบจะถูกออกแบบให้สามารถดำเนินเรื่องราวไปได้อย่างต่อเนื่องในลักษณะ Mission-based คือเมื่อจบภารกิจหนึ่ง ผู้เล่นจะสามารถรับภารกิจใหม่เพื่อเล่นต่อไปได้เรื่อยๆ โดยไม่มีจุดจบที่ตายตัว เพื่อรองรับการเล่นแบบปลายเปิด (Open-ended) และเพื่อใช้เป็นสถานการณ์ทดสอบหลักสำหรับ Pilot Test

3.5 การทดสอบและประเมินผล (System Testing and Evaluation)

การทดสอบระบบเป็นขั้นตอนสำคัญเพื่อรับประกันคุณภาพของต้นแบบ โดยจะแบ่งออกเป็น 2 ระยะหลัก เพื่อวัดผลเกณฑ์ชี้วัดความสำเร็จที่กำหนดไว้ใน Section 1.4.3

1. การทดสอบเชิงหน้าที่ (Functional Testing): เป็นการทดสอบเพื่อรับประกันความถูกต้องของโมดูลหลัก โดยเฉพาะ Rules Engine (เพื่อวัดผล Rule Adherence และ State Integrity). จะใช้วิธีการเขียน Unit Test ซึ่งเป็นการสร้างชุดคำสั่งอัตโนมัติเพื่อทดสอบการทำงานของฟังก์ชันต่างๆ ในสถานการณ์จำลอง (Scripted Scenarios) โดยอ้างอิงตามกฎ Lite 5e (ภาคผนวก ก). ตัวอย่าง Test Case เช่น

- 1) test_attack_roll(phys_bonus=2, target_ac=14) ต้องคืนค่า "Hit" ก็ต่อเมื่อผลทอย $d20 \geq 12$.
- 2) test_critical_hit_damage(weapon_die='1d6', phys_bonus=2): ต้องคืนค่าความเสียหายเป็นสองเท่าของลูกเต๋า + bonus เมื่อผลทอย $d20 = 20$.
- 3) test_skill_check(stat_bonus=1, dc=13) ต้องคืนค่า "Success" ก็ต่อเมื่อผลทอย $d20 \geq 12$.
- 4) test_hp_update(initial_hp=20, damage=7) ต้องอัปเดตค่า HP ใน GameState เป็น 13 อย่างถูกต้อง. แนวทางนี้มีความสำคัญอย่างยิ่งในการรับประกันว่าตรรกะหลักของเกมทำงานได้ถูกต้อง 100% ตามเกณฑ์ Rule Adherence และ State Integrity ก่อนนำไปบูรณาการกับ LLM

2. การทดลองใช้งานระบบ (Pilot Test) หลังจากบูรณาการทุกส่วนเป็น Prototype-B แล้ว จะมีการจัดทดลองใช้งานระบบกับกลุ่มเป้าหมาย 2-3 คน (คัดเลือกให้มีพื้นฐานหลากหลาย เช่น ผู้คุ้นเคยเกม

กระดานแต่ไม่เคยเล่น D&D และผู้ไม่เคยเล่นเลย) เพื่อ ประเมินประสบการณ์ใช้งานจริงและวัดผล
เกณฑ์ความสำเร็จจากมุมมองผู้ใช้

กระบวนการทดสอบจะแบ่งเป็น 3 ช่วง: 1) Onboarding, 2) ทดลองเล่น Scenario, และ 3)
Feedback Collection

1) แบบสอบถามความพึงพอใจ (Likert Scale 1-5) ใช้ประเมินเกณฑ์ชี้วัดหลักจากมุมมองผู้ใช้
ตัวอย่างคำถามเช่น

Q1 (Rule Adherence Perception): "การตัดสินใจผลของการกระทำ (เช่น การโจมตี) รู้สึกว่า
'ยุติธรรม' และ 'สอดคล้อง' กับกฎ Lite 5e หรือไม่?"

Q2 (State Integrity Perception) "ข้อมูลตัวละคร (HP, Zone) บนหน้าจอ 'ถูกต้อง' และ 'อัปเดต
ทันที' หรือไม่?"

Q3 (Flow Management) "การเล่นดำเนินไป 'ราบรื่น' หรือไม่? มีช่วงที่ 'ติดขัด' หรือ 'ไม่รู้จะ
ทำอะไรต่อ' หรือไม่?"

Q4 (Narrative - Path A) "คำบรรยายผลลัพธ์ของการกระทำตามกฎ (เช่น ผลโจมตี) 'ชัดเจน'
หรือไม่?"

Q5 (Narrative - Path B) "เมื่อลองทำอะไรนอกเหนือกฎ (เช่น ตำรวจ) คำตัดสิน/คำบรรยาย
ของ AI 'สมเหตุสมผล' หรือไม่?"

Q6 (Usability) "UI (หน้าจอต่างๆ) 'ใช้งานง่าย' หรือไม่?"

2) การสัมภาษณ์สั้นๆ (Qualitative Feedback) ส่วนนี้สำคัญมาก เพื่อเข้าใจ เหตุผล เบื้องหลัง
คะแนน และเปรียบเทียบมุมมองระหว่างผู้เล่นที่มีประสบการณ์ต่างกัน.

ทั้งนี้ การประเมิน Usability จะมุ่งเน้น UI หลัก ไม่รวมฟังก์ชันเสริม ASR/TTS. ผลตอบรับทั้งหมดจะ
ถูกนำมาวิเคราะห์เพื่อสรุปผลความสำเร็จของโครงการเทียบกับเกณฑ์ใน Section 1.4.3 และเป็น
แนวทางปรับปรุงต่อไป

บทที่ 4 ผลการดำเนินงานและการประเมินผล

ในบทนี้จะนำเสนอผลการดำเนินงานในส่วนของ Prototype-A ซึ่งมุ่งเน้นการพัฒนาและทดสอบระบบแกนหลัก (Core Mechanics) ของ AI Dungeon Master โดยเฉพาะส่วนของ Rules Engine และ Game State Management เพื่อยืนยันความถูกต้องของตรรกะเกม (Rule Adherence) ก่อนการบูรณาการกับส่วนอื่นๆ

4.1 ภาพรวมการพัฒนาโมดูลหลัก

เพื่อให้ระบบมีความยืดหยุ่นและตรวจสอบความถูกต้องได้ง่าย (Testability) คณะผู้จัดทำได้ดำเนินการพัฒนาโครงสร้างซอฟต์แวร์ตามสถาปัตยกรรม Two-Path โดยแบ่งส่วนประกอบสำคัญออกเป็น โมดูลย่อยที่มีหน้าที่ชัดเจน (Separation of Concerns) ดังนี้

1. โครงสร้างไฟล์และการจัดเก็บข้อมูล (Project Structure) ระบบถูกพัฒนาด้วยภาษา Dart โดยมีการจัดระเบียบไฟล์เพื่อแยกส่วนตรรกะ (Logic) ออกจากส่วนข้อมูล (Data) ดังแสดงในแผนภาพโครงสร้างไฟล์การทดสอบระบบเป็นขั้นตอนสำคัญเพื่อรับประกันคุณภาพของต้นแบบ โดยจะแบ่งออกเป็น 2 ระยะเวลาหลัก เพื่อวัดผลกระทบชีวิตความสำเร็จที่กำหนดไว้ใน Section 1.4.3

1) lib/logic/: บรรจุ Rules Engine ซึ่งทำงานแบบ Stateless Machine รับผิดชอบการคำนวณตัวเลขและการบังคับใช้กฎ Lite 5e ทั้งหมด

2) lib/models/: บรรจุ Game State Tracker ซึ่งทำหน้าที่เป็น Single Source of Truth ของระบบ เก็บสถานะตัวละครและศัตรูอย่างเป็นระบบ

3) lib/router/: เตรียมโครงสร้างสำหรับระบบ Router เพื่อแยกแยะเจตนาของผู้เล่น (Fixed vs Creative Action)

2. การพัฒนา Rules Engine (Stateless Logic) โมดูล Rules Engine ถูกพัฒนาให้เป็นชุดของ "ฟังก์ชันบริสุทธิ์" (Pure Functions) ที่รับข้อมูลนำเข้าและส่งผลลัพธ์ออกไปโดยไม่มีการเปลี่ยนแปลงสถานะภายในตัวเอง (Side Effects) เพื่อให้ผลลัพธ์มีความแน่นอน (Deterministic) และป้องกันปัญหา Hallucination จาก AI โดยครอบคลุมฟังก์ชันหลักดังนี้

1) rollDice: ระบบสุ่มตัวเลขที่รองรับรูปแบบลูกเต๋ามาตรฐาน (d20, d6, etc.)

2) resolveAttack: ระบบคำนวณผลการต่อสู้ที่รองรับการโจมตีระยะใกล้/ไกล (Zone-based) และการคำนวณความเสียหายตามประเภทอาชีพ

3) resolveCheck: ระบบตรวจสอบความสำเร็จของการใช้ทักษะเทียบกับค่าความยาก (DC)

4.2 ผลการทดสอบเชิงหน้าที่ของ Rules Engine (Functional Testing Results)

เพื่อเป็นการยืนยันความถูกต้องของกลไกเกม (Rule Adherence) และความสมบูรณ์ของสถานะ (State Integrity) คณะผู้จัดทำได้ดำเนินการทดสอบโมดูล Rules Engine ด้วยวิธีการ Unit Testing โดยครอบคลุมกรณีทดสอบ (Test Cases) ที่สำคัญตามข้อกำหนดของ Lite 5e ทั้งสิ้น 12 กรณี โดยมีรายละเอียดผลการทดสอบดังนี้

4.2.1 ผลการทดสอบกลไกพื้นฐาน (Core Mechanics Verification)

ส่วนนี้ทดสอบความถูกต้องของการคำนวณพื้นฐาน (Fundamental Math) ของระบบ D&D

1. Attack Roll Calculation: ระบบสามารถนำค่าผลรวมลูกเต๋า (d20) มาบวกกับค่าโบนัสสถานะ (Stat Modifier) ได้ถูกต้อง (เช่น Roll 10 + PHYS 2 = 12)
2. Armor Class (AC) Logic: ระบบสามารถเปรียบเทียบค่าผลรวมการโจมตี (Attack Total) กับค่าป้องกัน (AC) ของเป้าหมายเพื่อตัดสินผล Hit/Miss ได้อย่างแม่นยำ (Hit เมื่อ $Total \geq AC$)
3. Damage Application: เมื่อเกิดการโจมตีสำเร็จ (Hit) ระบบสามารถคำนวณความเสียหายและลดค่าพลังชีวิต (HP) ของเป้าหมายลงได้ถูกต้องตามจำนวนความเสียหายที่เกิดขึ้นจริง

4.2.2 ผลการทดสอบตรรกะเฉพาะอาชีพ (Class-Specific Logic)

ส่วนนี้ทดสอบความสามารถของระบบในการแยกแยะและปรับเปลี่ยนการคำนวณตามบทบาท (Role) ของตัวละคร

1. Fighter: ระบบเลือกใช้ค่าสถานะ PHYS และลูกเต๋าความเสียหาย 1d8 (Longsword) ได้ถูกต้อง
2. Paladin: ระบบเลือกใช้ค่าสถานะ PHYS และลูกเต๋าความเสียหาย 2d6 (Greatsword) ได้ถูกต้อง
3. Mage: ระบบสามารถปรับเปลี่ยนการโจมตีตามประเภทได้ถูกต้อง โดยเลือกใช้ค่าสถานะ MENT และลูกเต๋า 1d10 สำหรับเวทมนตร์ (Ranged Spell) และปรับค่า Modifier ได้ถูกต้อง

4.2.3 ผลการทดสอบตรรกะระยะและโซน (Zone & Range Logic)

ส่วนนี้ทดสอบความถูกต้องของการบังคับใช้กฎการเคลื่อนที่และระยะโจมตี (Zone-based Rules)

1. Melee Constraint: ระบบสามารถ บล็อก (Block) การโจมตีระยะประชิด (Melee) ได้ หากผู้โจมตีและเป้าหมายอยู่คนละโซน (Distance > 0) พร้อมแจ้งเตือนข้อผิดพลาด
2. Ranged Disadvantage: ระบบสามารถตรวจสอบระยะห่างและ บังคับใช้กฎเสียเปรียบ (Disadvantage) ได้อัตโนมัติ เมื่อมีการโจมตีระยะไกลข้าม 2 โซน (เช่น Near ไป Far) โดยทำการทอยลูกเต๋า 2 ครั้งและเลือกค่าที่ต่ำกว่า

4.2.4 ผลการทดสอบกรณีขอบเขตและข้อผิดพลาด (Edge Cases & Robustness)

ส่วนนี้ทดสอบความทนทานของระบบต่อสถานการณ์ที่ไม่ปกติ

1. Dead Target Protection: ระบบสามารถป้องกันการโจมตีใส่เป้าหมายที่เสียชีวิต (Dead) หรือหมดสติ (Unconscious) ได้อย่างถูกต้อง
2. Case Insensitivity: ระบบมีความยืดหยุ่นในการรับคำชื่ออาชีพ (Role) โดยสามารถระบุตัวตนได้ถูกต้องแม้จะมีการพิมพ์ตัวพิมพ์เล็ก-ใหญ่ผสมกัน (เช่น "fiGHTer" = Fighter)

4.2.5 สรุปผลการทดสอบ

จากการรันชุดทดสอบอัตโนมัติ พบว่าระบบ ผ่านการทดสอบครบถ้วนทั้ง 12 กรณี (All tests passed) ซึ่งเป็นการยืนยันว่า Rules Engine ที่พัฒนาขึ้นมีความถูกต้องแม่นยำ 100% ตามข้อกำหนดของกติกา Lite 5e และมีความพร้อมสมบูรณ์สำหรับการนำไปใช้งานในขั้นตอนต่อไป

เอกสารอ้างอิง

บทความในรายงานการประชุมทางวิชาการและวารสาร

1. Triyason, T., et al., 2023, "**Exploring the Potential of ChatGPT as a Dungeon Master in Dungeons & Dragons Tabletop Game**", Proceedings of the 2023 International Conference on Digital Image and Multimedia (ICODIM), 1-3 December, Chiang Mai, Thailand, pp. 1-6.
2. Jørgensen, N. H., Tharmabalan, T., Aslan, M., & Merritt, T., 2024, "**Evaluating Multi-Agent Architectures for an AI Game Master in Dungeons & Dragons**", Extended Abstracts of the 2024 CHI Conference on Human Factors in Computing Systems, 11-16 May, Honolulu, HI, USA.
3. Lanchantin, J., et al., 2024, "**You Have Thirteen Hours in Which to Solve the Labyrinth: Enhancing AI Game Masters with Function Calling**", arXiv preprint arXiv:2409.06949.
4. Vaswani, A., et al., 2017, "**Attention Is All You Need**", Advances in Neural Information Processing Systems 30 (NIPS 2017), Long Beach, CA, USA, pp. 5998-6008.
5. Radford, A., et al., 2022, "**Robust Speech Recognition via Large-Scale Weak Supervision**", Proceedings of the 39th International Conference on Machine Learning (ICML), 17-23 July, Baltimore, Maryland, USA, pp. 18076-18093.
6. Sakellaridis, P., 2024, **AI Dungeon Master for DnD Railroads Players**, [Online], Available: <https://www.wargamer.com/dnd/ai-railroads-players> [2025, October 19].
7. Maassen, H., 2019, **D&D 5e Lite**, [Online], Available: <https://haraldmaassen.com/blog/2019/06/02/dd-5e-lite/> [2025, October 19].
8. Babakcode, 2024, **flutter_gemini 3.0.0**, [Online], Available: https://pub.dev/packages/flutter_gemini [2025, October 19].

ภาคผนวก ก

ชุดกติกาย่อ Lite 5e (ฉบับสำหรับทีลีส)

ชุดกติกาย่อ Lite 5e (ฉบับสำหรับโครงงาน)

บทนำ

ยินดีต้อนรับนักผจญภัย! คู่มือกติกาที่ถูกออกแบบมาสำหรับผู้เล่นใหม่ และเพื่อวัตถุประสงค์ของโครงงานนี้โดยเฉพาะ ระบบของเราได้รับแรงบันดาลใจจาก Dungeons & Dragons ฉบับที่ 5 แต่ได้ถูกปรับให้เรียบง่ายขึ้น เพื่อให้:

- ง่ายต่อการเรียนรู้
- เล่นได้อย่างรวดเร็ว
- เหมาะสมกับการโต้ตอบกับ AI Dungeon Master (LLM)

ไม่ว่าคุณจะเป็นนักรบที่เหี้ยมฉาด, นักบวชผู้ร้องขอพลังจากทวยเทพ, หรือจอมเวทผู้ควบคุมความเป็นจริง กฎเหล่านี้จะมอบโครงสร้างที่ยืดหยุ่นสำหรับการเล่นของคุณ ในขณะที่ยังคงรักษาอิสระในการดำเนินเรื่องราวไว้

วิธีการเล่น (How to Play)

หัวใจหลักของเกมนี้เรียบง่าย:

1. LLM/DM บรรยายสถานการณ์ (เช่น “ก๊อบลินตัวหนึ่งกระโดดออกมาจากหลังถ้ำ!”)
2. ผู้เล่นบอกสิ่งที่คุณต้องการจะทำ (เช่น “ฉันจะใช้ดาบฟันไปที่ก๊อบลิน!”)
3. ระบบทอยลูกเต๋าเพื่อดูว่าคุณทำสำเร็จหรือไม่ (d20 + PHYS หรือ d20 + MENT เทียบกับ AC/DC)
4. ผลลัพธ์จะถูกบรรยายออกมา (เช่น “ดาบของคุณฟันเข้าอย่างจัง! ก๊อบลินได้รับความเสียหาย 7 หน่วย”)

พื้นฐานเรื่องลูกเต๋า (Dice Basics)

เราจะใช้ระบบ d20 เป็นหลัก:

- การตรวจสอบทักษะ (Skill Checks): d20 + ค่าโบนัสสถานะ เทียบกับ ค่าความยาก (DC)
- การโจมตี (Attack Rolls): d20 + ค่าโบนัสสถานะ เทียบกับ ค่าความป้องกัน (AC) ของศัตรู
- การคำนวณความเสียหาย (Damage Rolls): ทอยลูกเต๋าลูกที่เล็กกว่า (d6, d8, etc.)

การทอยพิเศษ:

- ใต้แต้ม 20 (Natural 20): สำเร็จโดยอัตโนมัติ (Critical Success) และสร้างความเสียหายสองเท่า (สำหรับการโจมตี)
- ใต้แต้ม 1 (Natural 1): ล้มเหลวโดยอัตโนมัติ (Critical Failure) และอาจมีผลเสียตามมา

ค่าสถานะและค่าโบนัส (Stats & Modifiers)

เราได้รวมค่าความสามารถทั้ง 6 ของ D&D แบบดั้งเดิมให้เหลือเพียง 3 ค่าสถานะหลัก:

ค่าสถานะ	ครอบคลุม	ตัวอย่างการกระทำ
กายภาพ (PHYS)	พลัง, ความคล่องแคล่ว, ความทนทาน	โจมตี, ปีนป่าย, หลบหลีก, ผลัก
สติปัญญา (MENT)	ความฉลาด, เชว่นปัญญา, การรับรู้	ค้นหาความรู้, สังเกตกับดัก, แก้ปริศนา
สังคม (SOC)	เสน่ห์, การโน้มน้าวใจ	โน้มน้าวใจ, โทก, ช่มชู้

- แต่ละค่าสถานะจะมี ค่าโบนัส (Modifier) ตั้งแต่ -2 ถึง +5 ซึ่งจะนำไปบวกกับการทอยลูกเต๋า

พลังชีวิต (Hit Points - HP)

- ตัวละครทั้งหมดเริ่มต้นที่ 20 HP ที่เลเวล 1
- ความเสียหายจะลดค่า HP
- HP เหลือ 0 = หหมดสติ (Unconscious)
- หากไม่ได้รับการรักษา อาจนำไปสู่ความตายได้

ระบบการต่อสู้ (Combat System)

การต่อสู้เป็นแบบ ผลัดกันเล่น (Turn-based) และใช้ระบบ โซน (Zone-based)

ลำดับการเล่น (Turn Order)

- ผู้เล่น → ศัตรู → ผู้เล่น → ศัตรู ... ดำเนินไปเรื่อยๆ จนกว่าฝ่ายใดฝ่ายหนึ่งจะพ่ายแพ้หรือล่าถอย

การกระทำในแต่ละรอบ (Actions per Turn)

- ในรอบของคุณ คุณสามารถทำ 1 การกระทำ (Action) + 1 การเคลื่อนที่ (Move):
 - การกระทำ (Action): โจมตี (ระยะประชิด/ระยะไกล), ร่ายเวท (ถ้ามี), ใช้ความสามารถพิเศษ, ใช้ไอเทม (ดื่มยา, โยนขวด), มีปฏิสัมพันธ์กับสิ่งแวดล้อม (ถีบประตู, ดึงคันโยก)
 - การเคลื่อนที่ (Move): เคลื่อนที่ไปยังโซนอื่น (ดูหัวข้อถัดไป)

การเคลื่อนที่ (Movement)

เราไม่ใช้ตารางการต่อสู้ แต่จะแบ่งพื้นที่ออกเป็น โซน:

- โซนเดียวกัน (Same Zone): สามารถโจมตีระยะประชิดและระยะไกลได้
- โซนติดกัน (Adjacent Zone): สามารถโจมตีระยะไกลได้ แต่ระยะประชิดไม่ได้

- โซนไกล (Far Zone): สามารถโจมตีระยะไกลได้ แต่จะมีค่าเสียเปรียบ (Disadvantage - อาจจะยังไม่ implement ในต้นแบบ)
- ผู้เล่นสามารถเคลื่อนที่ได้ 1 โซนต่อรอบ เป็นการเคลื่อนที่ (Move)

การโจมตี (Attacking) การโจมตีแบ่งเป็น 2 ประเภทตามอาวุธที่ใช้:

1. การโจมตีระยะประชิด (Melee Attack):

- เงื่อนไข: ผู้โจมตีและเป้าหมายต้องอยู่ โซนเดียวกัน (Same Zone)
- สูตรทอย: $d20 + \text{PHYS}$ vs AC
- ความเสียหาย: $\text{Weapon Dice} + \text{PHYS}$

2. การโจมตีระยะไกล/เวทมนตร์ (Ranged/Spell Attack):

- เงื่อนไข: โจมตีได้ทุกโซน (หากเป้าหมายอยู่ห่างกัน 2 โซน เช่น Near ไป Far จะเสียเปรียบ/Disadvantage)
- สูตรทอย: $d20 + \text{MENT}$ (สำหรับเวท) หรือ $d20 + \text{PHYS}$ (สำหรับธนู) vs AC
- ความเสียหาย: $\text{Spell/Weapon Dice} + \text{MENT}$ (สำหรับเวท) หรือ $+ \text{PHYS}$ (สำหรับธนู)

3. การโจมตีคริติคอล (Critical Hit):

- เงื่อนไข: เมื่อผลการทอยลูกเต๋าโจมตี (Attack Roll) ออกหน้า 20 (Natural 20)
- ผลลัพธ์: การโจมตีจะโดนเป้าหมายทันที (Auto Hit) และ ความเสียหาย (Damage Dice) จะถูกคูณสอง (Double Dice) ก่อนบวกค่าโบนัส
- ตัวอย่าง: ปกติ $1d8 + 3 \rightarrow$ คริติคอลจะเป็น $2d8 + 3$

ตัวอย่างอาชีพ (Example Classes)

- นักรบ (Fighter):
 - HP: 20 | AC: 16 (Chainmail)
 - Attack (Melee): ดาบยาว (Longsword) $\rightarrow 1d8 + \text{PHYS}$
 - Ability: Second Wind $\rightarrow$ รักษา HP $1d10 + 1$ (1 ครั้ง/การต่อสู้)
- พาลาดิน (Paladin):
 - HP: 20 | AC: 16 (Chainmail)
 - Attack (Melee): ดาบใหญ่ (Greatsword) $\rightarrow 2d6 + \text{PHYS}$
 - Ability: Smite $\rightarrow$ เพิ่มความเสียหาย $2d8$ (2 ครั้ง/วัน)
 - Ability: Lay on Hands $\rightarrow$ รักษา HP $1d8 + \text{MENT}$ (1 ครั้ง/วัน)
- นักบวช (Cleric):
 - HP: 18 | AC: 18 (Plate + Shield)

- Attack (Melee): ค้อนตีก (Warhammer) → 1d8 + PHYS
- Ability: Pray (ทอย MENT เทียบกับ DC 13) → เลือกผล: รักษาพันธมิตร (1d8 + MENT) หรือ โจมตีศัตรู (1d8 + MENT Radiant Dmg)
- จอมเวท (Mage):
 - HP: 15 | AC: 12 (Robes + Mage Armor)
 - Attack (Ranged Spell): คทาไฟ (Firebolt) → 1d10 + MENT (ระยะไกล)
 - Attack (Melee): ไม้เท้า (Quarterstaff) → 1d6 + PHYS (ระยะประชิด - กรณีถูกเงิน)
 - Spells: 2 ครั้ง/วัน (รูปแบบอิสระ, LLM ทำหน้าที่กำหนด DC และผลกระทบ)

การพักผ่อนและเวลา (Rest & Time)

- พักสั้น (Short Rest):
 - ระยะเวลา: 1 ชั่วโมง
 - ฟื้นฟู HP 1d8 + ค่าโบนัส (ถ้ามี)
 - ฟื้นฟูความสามารถที่ใช้ได้จำกัด (เช่น Second Wind)
- พักยาว (Long Rest):
 - ระยะเวลา: 8 ชั่วโมง
 - ฟื้นฟู HP ทั้งหมด
 - ฟื้นฟูเวทมนตร์และความสามารถทั้งหมด
 - ต้องทำในที่ที่ปลอดภัย

สถานะผิดปกติ (Conditions)

- หมดสติ (Unconscious): HP เหลือ 0, ไม่สามารถกระทำการใดๆ ได้
- มึนงง (Stunned): ขำรอบของตัวเองในเทิร์นถัดไป
- ถูกพันธนาการ (Restrained): การทอยเต๋าที่ใช้ค่า PHYS จะเสียเปรียบ (Disadvantage)
- ตาย (Dead): ออกจากเกม เว้นแต่จะถูกชุบชีวิต

การเพิ่มระดับ (Leveling Up)

(หมายเหตุ: เป็นทางเลือกสำหรับต้นแบบ อาจยังไม่ implement)

- +5 HP ต่อเลเวล
- เลือกระหว่าง: +1 ค่าสถานะ หรือ เพิ่มการใช้ความสามารถ/เวทมนตร์ใหม่
- ระบบ XP อย่างง่าย: 250 XP ต่อเลเวล